

# Procesamiento de Lenguaje Natural & Visión Computacional | Tarea #2

*César M. Aguirre Calzadilla*

*Centro de Investigación en Matemáticas*

***Catedráticos:***

12 de octubre de 2025

# Tabla de contenidos

|                                                                        | Página    |
|------------------------------------------------------------------------|-----------|
| <b>Recursos computacionales</b> . . . . .                              | <b>4</b>  |
| <b>Modelos de Lenguaje</b> . . . . .                                   | <b>4</b>  |
| <b>Perplejidad</b> . . . . .                                           | <b>5</b>  |
| PPL en modelos de Redes Neuronales . . . . .                           | 6         |
| <b>Parte A   Generación de texto</b> . . . . .                         | <b>7</b>  |
| Limpieza para corpus generativo y para corpus clasificadorio . . . . . | 8         |
| Exploratorio del corpus . . . . .                                      | 8         |
| <b>Parte A   RNN</b> . . . . .                                         | <b>11</b> |
| Formulación probabilística de los Modelos de Lenguaje (ML) . . . . .   | 11        |
| Arquitectura de las RNN . . . . .                                      | 12        |
| Arquitectura Utilizada e Hiperparámetros . . . . .                     | 13        |
| Estructura del Modelo . . . . .                                        | 13        |
| Hiperparámetros de Entrenamiento . . . . .                             | 14        |
| Resultados . . . . .                                                   | 14        |
| Conclusiones . . . . .                                                 | 15        |
| <b>Parte A   LSTM</b> . . . . .                                        | <b>17</b> |
| Compuerta del Olvido (forget gate) . . . . .                           | 18        |
| Compuerta de Adición (add gate) . . . . .                              | 19        |
| Compuerta de Salida (output gate) . . . . .                            | 19        |
| Arquitectura Utilizada e Hiperparámetros . . . . .                     | 19        |
| Estructura del Modelo . . . . .                                        | 20        |
| Hiperparámetros de Entrenamiento . . . . .                             | 20        |
| Resultados . . . . .                                                   | 20        |
| Conclusiones . . . . .                                                 | 22        |
| <b>Parte A   GRU</b> . . . . .                                         | <b>23</b> |
| La célula GRU y sus compuertas . . . . .                               | 23        |
| Arquitectura Utilizada e Hiperparámetros . . . . .                     | 24        |
| Arquitectura del Modelo GRU . . . . .                                  | 24        |
| Hiperparámetros de Entrenamiento . . . . .                             | 25        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| Resultados . . . . .                                                       | 25        |
| Conclusiones . . . . .                                                     | 26        |
| <b>Introducción a los Transformers y su relación con los LLM . . . . .</b> | <b>28</b> |
| El cambio de paradigma . . . . .                                           | 28        |
| <i>Self-Attention</i> . . . . .                                            | 29        |
| <i>Multi-head Attention</i> . . . . .                                      | 30        |
| Transformers populares . . . . .                                           | 30        |
| GPT (Generative Pretrained Transformer) . . . . .                          | 30        |
| LLaMA (Large Language Model Meta AI) . . . . .                             | 31        |
| DeepSeek . . . . .                                                         | 31        |
| Otros modelos influyentes . . . . .                                        | 32        |
| <b>Parte A   TinyLLaMA-1.1B . . . . .</b>                                  | <b>33</b> |
| Resultados . . . . .                                                       | 33        |
| Conclusiones . . . . .                                                     | 34        |
| <b>Parte A   Mistral-7B LoRA . . . . .</b>                                 | <b>36</b> |
| Mistral-7B LoRA . . . . .                                                  | 36        |
| Resultados . . . . .                                                       | 38        |
| Conclusiones . . . . .                                                     | 40        |
| <b>Parte B   CNN . . . . .</b>                                             | <b>42</b> |
| Arquitectura CNN . . . . .                                                 | 42        |
| Resultados . . . . .                                                       | 42        |
| Visualización de Modelos . . . . .                                         | 42        |
| Conclusiones . . . . .                                                     | 44        |
| <b>Parte B   RNN . . . . .</b>                                             | <b>46</b> |
| Arquitectura RNN . . . . .                                                 | 46        |
| Resultados . . . . .                                                       | 47        |
| Visualización de Modelos . . . . .                                         | 47        |
| Conclusiones . . . . .                                                     | 48        |
| <b>Parte B   LSTM . . . . .</b>                                            | <b>50</b> |
| Arquitectura LSTM . . . . .                                                | 50        |
| Un Modelo de Representación Global . . . . .                               | 50        |
| Conclusiones . . . . .                                                     | 51        |
| Conclusiones . . . . .                                                     | 52        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| Visualización de Modelos . . . . .                         | 52        |
| <b>Parte B   GRU . . . . .</b>                             | <b>55</b> |
| Resultados . . . . .                                       | 55        |
| Conclusiones . . . . .                                     | 56        |
| Visualización de Modelos . . . . .                         | 56        |
| <b>Transformers para clasificación de textos . . . . .</b> | <b>59</b> |
| Clasificación de texto con Transformers . . . . .          | 59        |
| Contraste con el enfoque generativo . . . . .              | 60        |
| <b>Parte B   BETO . . . . .</b>                            | <b>61</b> |
| Visualización de Modelos . . . . .                         | 61        |

## Recursos computacionales

Para esta tarea, nos apoyamos del Laboratorio de Supercómputo de CIMAT. El servidor de este Laboratorio cuenta con dos unidades NVIDIA TITAN RTX, con 24 GB de VRAM por GPU. Además, cuenta con 12.3 nodos y 12.1 con conda. El acceso es remoto, vía SSH y trabajamos con nodos computacionales sin acceso a internet. La descarga de los modelos Transformer se realizaron primero a mi computadora y luego se subieron al Laboratorio.

Esto nos permitió manejar y finetunar modelos más grandes de lo que mi computadora puede soportar, como lo fueron Mistral-7B o LLaMA. En mi caso, los modelos corridos en mi PC fueron las arquitecturas de redes neuronales recurrentes (RNN, LSTM y GRU). Mi computadora cuenta con una NVIDIA GeForce GTX 1650, de 4GB de VRAM y 12.4 nodos.

Entre las cosas que aprendimos, aparte de lo solicitado en la tarea, fue el manejo de SLURM y de comandos para servidores como el de CIMAT.

Los tiempos de ejecución para los modelos de Transformers en general fueron rápidos, aproximadamente 10 por prueba. En el caso de los modelos de redes neuronales recurrentes, los tiempos de ejecución en mi computadora tardaron máximo 30 minutos por modelo, dependiendo de los hiperparámetros.

## Modelos de Lenguaje

Un Modelo de Lenguaje, o LM por sus siglas en inglés, es un sistema probabilístico que asigna una probabilidad a una secuencia de palabras, o bien, predice la probabilidad de aparición de la siguiente palabra dado un contexto previo. Formalmente, un modelo de lenguaje define una distribución sobre secuencias  $\omega_1, \omega_2, \dots, \omega_n$ , de la forma:

$$P(\omega_1, \omega_2, \dots, \omega_n) = \prod_{i=1}^n P(\omega_i \mid \omega_1, \omega_2, \dots, \omega_{i-1}),$$

Donde cada término  $P(\omega_i \mid \omega_{<i})$  representa la probabilidad condicional de la palabra  $\omega_i$  dado su historia previa  $\omega_{<i}$ .

Los modelos de lenguaje tienen una larga tradición en el procesamiento de lenguaje natural. Los primeros enfoques se basaron en modelos de n-gramas, que utilizan la suposición de Markov, i.e. la probabilidad de una palabra depende únicamente de las últimas  $n - 1$  palabras de contexto. Por ejemplo, un modelo de bigramas:

$$P(\omega_i \mid \omega_{<i}) \approx P(\omega_i \mid \omega_{<i-1})$$

Estos modelos, aunque eran sencillos y computacionalmente eficientes, presentaban limitaciones claras al no poder capturar dependencias de largo alcance, además de sufrir de "esparsidad", es decir, muchas secuencias posibles nunca aparecían en los datos de entrenamiento. Es así como el desarrollo de técnicas de suavizado como Laplace o Kneser-Ney permitieron mejorar el desempeño de los LM, pero no se resolvió por completo en cuanto a escasez de datos.

Tras el auge del Deep Learning demostrado en la última década, los modelos de redes neuronales reemplazaron a los modelos de n-gramas, pues estos eran capaces de aprender representaciones continuas y modelar dependencias mucho más largas de texto. Al poco tiempo, los Transformers cambiaron el paradigma y permitieron ampliar dicha capacidad, aprovechando del mecanismo de self-attention para capturar relaciones globales en la secuencia, y a entrenar a muy gran escala llegando a lo que ahora conocemos como Large Language Models o LLM, como lo son GPT, BERT, LLama y todo el abanico de sucesores.

El propósito central de un LM es estimar la probabilidad de una secuencia de palabras, lo cual lo convierte en una herramienta sumamente útil para una enorme cantidad de tareas. Sin embargo, dado que todo LM asigna distribuciones de probabilidad sobre texto, su calidad se mide en qué tan bien predice secuencias reales del idioma. El criterio intrínseco más utilizado para esta tarea de evaluación es la perplejidad.

## Perplejidad

La Perplejidad (PPL) es la métrica estándar para evaluar modelos de lenguaje, especialmente en tareas de predicción de secuencias. De manera intuitiva, PPL mide qué tan sorprendido está un modelo frente a un conjunto de validación, cuanto mejor predice las palabras reales que aparecen en el texto, menor será su perplejidad.

Formalmente, para un conjunto de prueba  $W = \omega_1, \omega_2, \dots, \omega_N$ , la perplejidad se define como:

$$\text{PPL}(W) = \exp \left( -\frac{1}{N} \sum_{i=1}^N \log p(\omega_i | \omega_{<i}) \right)$$

La expresión anterior equivale a:

$$\text{PPL}(W) = \exp \left( H(p, q) \right)$$

Donde  $p(\omega_i | \omega_{<i})$  es la probabilidad asignada por el modelo para la palabra  $\omega_i$ , dado el contexto anterior.

Esta definición puede interpretarse como la media geométrica inversa de las probabilidades de que el modelo asigna a las palabras correctas. En consecuencia:

- Una PPL baja implica que el modelo asigna alta probabilidad a las palabras reales, por lo que hay una mejor capacidad de predicción.
- Una PPL alta implica que el modelo se sorprende con frecuencia, entonces predice mal el texto.

Así entonces, la perplejidad equivale al número promedio de opciones equiprobables entre las que el modelo “cree” tener que elegir en cada paso. Por ejemplo, una  $PPL = 3$  significa que el modelo se comporta como si en promedio tuviera que escoger entre tres palabras posibles en cada posición.

La perplejidad está directamente relacionada con la entropía cruzada de la distribución real frente a la estimada por el modelo. Es decir, minimizar la perplejidad equivale a maximizar la probabilidad del conjunto de validación. Por esta razón, se utiliza ampliamente para comparar modelos de lenguaje entrenados con distintos algoritmos o configuraciones.

Es importante destacar que:

- No siempre correlaciona perfectamente con la calidad extrínseca de tareas como traducción automática o generación de texto, donde aspectos de coherencia semántica y adecuación estilística son también relevantes.
- La comparación entre modelos requiere que se use el mismo vocabulario y corpus de prueba; de lo contrario, las cifras no son conmensurables.
- Modelos con perplejidad baja pueden generar texto gramatical pero poco creativo (alta repetición), mientras que temperaturas de muestreo altas pueden producir lo opuesto: diversidad pero con mayor perplejidad.

## PPL en modelos de Redes Neuronales

En arquitecturas modernas como RNN, LSTM, GRU o Transformers, el cálculo de la perplejidad se implementa de manera directa a partir de la función de *cross-entropy*. dado que durante el entrenamiento se minimiza la entropía cruzada entre las etiquetas reales y la distribución del modelo, evaluar la perplejidad en el conjunto de validación es equivalente a medir el valor exponencial de dicha pérdida.

## Parte A | Generación de texto

Reune al menos 100 canciones de tu género de música favorito. Unifica todo en un solo archivo de texto (canciones.txt), separando cada canción con un delimitador claro. Por ejemplo: `<|startsong|>` y `<|endsong|>` en una línea.

Además, normaliza con codificación UTF-8, elimina ruido (cabeceras repetidas, metadatos, anuncios) y documenta tus decisiones de preprocesamiento.

Para esta tarea se reunieron canciones de la banda 'Twenty One Pilots'. Entre su discografía se encuentran más de 100 canciones, que en general combinan composiciones y estructuras del Hip Hop, Rap Rock, electropop y Synth-pop. En general es una banda que combina distintos géneros, ritmos y estructuras, pero pensé que mantener un corpus de una misma banda podría ser interesante.

Para no descargar de manera manual las canciones, utilizamos la API de Genius. Genius es una página web dedicada a la captura y exposición de letras de canciones, así como a la colaboración con fanáticos de las bandas para interpretar y dar explicación a fragmentos de las canciones. La API de esta página permite, de manera sencilla, acceder a su base de datos para sustraer las letras que te interesen. Puedes realizarlo por título de canción, o por álbum, así como por artista. La descarga de los datos es relativamente sencilla utilizando Python.

El proceso de descarga se automatizó mediante un script, utilizando la biblioteca `lyricsgenius`, la cual funciona como un cliente para la API de Genius. La implementación se dividió en varias fases clave para asegurar una recolección de datos robusta y limpia.

Primero, se configuró el acceso a la API con un token de autenticación. Para refinar la calidad de los datos desde el inicio, se establecieron parámetros específicos en el cliente de `lyricsgenius`:

- Se excluyeron términos como "Remix" y "Live" para evitar versiones alternativas de las canciones y enfocarse en las letras de estudio.
- Se activó la opción `remove_section_headers` para eliminar automáticamente las etiquetas estructurales de las letras (ej. [Chorus], [Verse]), lo cual representa un primer paso de preprocesamiento.

Además de canciones por álbum se añadieron a la lista de descargas canciones que son sencillos de la banda, como 'Heathens' y 'Level of Concern'. Toda la información recolectada, es decir, el título de la canción, el álbum y la letra, se almacenó en una estructura de datos utilizando Pandas.



Como último paso de limpieza en esta sección, se eliminaron posibles entradas duplicadas revisando el título de las canciones. El resultado fue un corpus consolidado por un archivo en CSV llamado: TOP\_lyrics\_hybrid.csv.

## Limpieza para corpus generativo y para corpus clasificatorio

Se tomó la decisión de realizar dos limpiezas de texto distinta: una para el caso de los modelos generativos y otra para el caso de los modelos clasificatorios. Esto se realizó así por una razón en particular, y es que en el caso generativo es importante mantener stop words y ciertas estructuras musicales como los paréntesis para coros, dentro del corpus. Esto es así porque, idealmente, queremos que nuestros modelos logren interpretar bien la estructura musical, ritmo y pacing tal cual está estructurado en la letra sin modificaciones.

Para el caso de los modelos clasificadores, se aplicará una limpieza más agresiva, quitando stop words, pasando todo a minúsculas y quitando arreglos que definan la estructura de canciones como lo son 'chorus' 'verse' o 'bridge'.

## Exploratorio del corpus

Se realizó un exploratorio sencillo del corpus. Comenzamos por revisar qué palabras son las más frecuentes en las canciones de Twenty One Pilots hasta el momento.

Destacan en el top 3 las palabras *don't*, *know*, *take*, *time* y *one*. Conociendo el contexto de la banda y el tipo de historias y narrativas que manejan en sus canciones, estas cinco palabras tienen mucho sentido. Se trata de una banda que mantiene una postura abierta respecto a los problemas de salud mental, la duda ante uno mismo, la incertidumbre de la vida y la oscuridad que puede llegar a habitar nuestra mente. El discurso interno que mantiene la banda nos refleja la duda, la ansiedad y la confusión. El término de *time* puede ser una señal de preocupación ante el paso del tiempo y cómo este influye en gran medida en los procesos personales y las luchas internas.

Palabras como *go*, *tell*, *need* y *save* pueden apuntar a un sentido de urgencia y la necesidad de acción. Son palabras de alta frecuencia, las cuales se alinean a la perfección con las letras que cuentan la lucha por seguir adelante o la necesidad de ayuda.

Creo que esto es un primer paso para el perfilado de autor en el ámbito de la lírica musical. Vemos en estas palabras el estilo conversacional y confesional de las letras compuestas por Tyler Joseph. Las canciones de Twenty One Pilots a menudo se sienten como un desgarramiento emocional y una confesión de dolor, peligro o ayuda. Entonces, creo que hace mucho sentido esta lista.

Podemos ver la gráfica de la ley de Zipf para las canciones de Twenty One Pilots en la figura 1.

Cuadro 1: Las 15 palabras más frecuentes (sin \*stopwords\*).

| Palabra | Frecuencia | Posición |
|---------|------------|----------|
| don't   | 627        | 1        |
| know    | 594        | 2        |
| take    | 300        | 3        |
| time    | 297        | 4        |
| one     | 288        | 5        |
| like    | 272        | 6        |
| go      | 241        | 7        |
| tell    | 227        | 8        |
| need    | 215        | 9        |
| 'cause  | 205        | 10       |
| save    | 204        | 11       |
| think   | 193        | 12       |
| say     | 191        | 13       |
| never   | 189        | 14       |
| 'ive    | 187        | 15       |

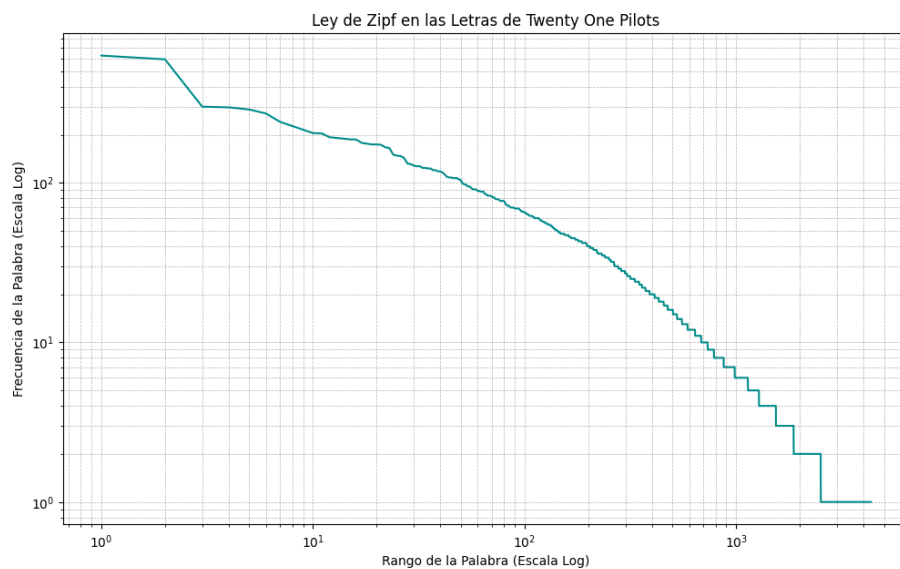


Figura 1: Gráfica de la ley de Zipf para letras de Twenty One Pilots.

De igual forma, podemos visualizar la frecuencia de las palabras de una manera más artística implementando una wordcloud.



Figura 2: Wordcloud de Twenty One Pilots.

## Parte A | RNN

Entrena los modelos con las canciunes y con las arquitecturas RNN/LSTM/GRU. A nivel caracter y a nivel palabra.

Los modelos de lenguaje (LM por sus siglas en inglés), buscan asignar probabilidades a secuencias de palabras o caracteres y, en particular, estimar la probabilidad del siguiente token dado el contexto previo. Durante décadas, esto se abordó con n-gramas; la métrica canónica es la perplejidad (PPL), la cual se definió como el exponencial de la entropía cruzada promedio, o de manera equivalente, es la inversa de la probabilidad media del corpus. Minimizar la perplejidad es maximizar la probabilidad de los datos, y cuanto menor es la PPL, mejor el modelo en términos de sorpresa promedio por token.

En ese sentido, las Redes Neuronales Recurrentes (RNN), introdujeron una forma distinta de tratar el tiempo. Es decir, las RNN comparten pesos a lo largo de la secuencia y actualizan un estado oculto que "transporta" la información entre pasos, permitiendo modelar dependencias más largas que las que capturan n-gramas. Este paradigma dio paso a variantes con compuertas, que veremos más adelante, como lo son las LSTM y GRU. Fueron la piedra de inicio que también dio el primer paso para llegar a lo que ahora conocemos como Transformers y que son la base para los LLM.

Aún así, las RNN y sus variantes siguen siendo fundamentales como baselines y por su valor didáctico, pues son la puerta de inicio para el aprendizaje por secuencias y el Procesamiento de Lenguaje Natural con modelos de Redes Neuronales.

## Formulación probabilística de los Modelos de Lenguaje (ML)

Sea  $\omega_1; T$  una secuencia. Un LM autoregresivo factoriza la probabilidad conjunta como:

$$p(\omega_{1:T}) = \prod_{t=1}^T p(\omega_t | \omega_{<t})$$

Esta descomposición habilita el aprendizaje autosupervisado, i.e. donde cada paso temporal proporciona su propio "objetivo", de tal modo que  $\omega_t$  se predice a partir de  $\omega_{<t}$ . El entrenamiento minimiza la entropía cruzada promedio (NLL), y la perplejidad se define como  $PPL = \exp(\text{CE Promedio})$ .

Un procedimiento práctico asociado es el *teacher forcing*: durante el entrenamiento, el modelo recibe como entrada el token verdadero del paso anterior y aprende a predecir el

siguiente.

## Arquitectura de las RNN

En una RNN "simple", cada entrada  $x_t$  (el embedding del token en  $t$ ), actualiza el estado oculto  $h_t$  con pesos compartidos.

En esta formulación, los parámetros de la red se organizan en tres matrices fundamentales que gobiernan el flujo de información:

- $W_{xh}$  (a menudo denotada  $U$ ) transforma la representación de entrada  $x_t$  hacia el espacio del estado oculto.
- $W_{hh}$  (denotada en algunos textos como  $W$ ) es la matriz recurrente que proyecta el estado oculto previo  $h_{t-1}$  y permite transportar la información a lo largo de la secuencia.
- $W_{ho}$  (o  $V$ ) mapea el estado oculto actual  $h_t$  hacia el espacio de salida, produciendo los logits sobre el vocabulario.

Estas matrices son compartidas en todos los pasos temporales, lo cual constituye la esencia de la recurrencia: el modelo reutiliza los mismos parámetros para procesar cualquier posición de la secuencia, logrando generalizar más allá de la ventana fija de los modelos n-grama.

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad o_t = W_{ho}h_t + b_o \quad \hat{y}_t = \text{Softmax}(o_t)$$

Aquí,  $\hat{y}_t$  es la distribución sobre el vocabulario para el próximo token. Esta arquitectura se desenrolla en el tiempo para visualizar el flujo de información y de gradientes a lo largo de las secuencias.

Acorde con Dan Jurafsky en el libro 'Speech and Language Processing', las capas ocultas de los pasos anteriores funcionan a manera de memoria, o de contexto. Se trata de una especie de sistema que procesa la información para posteriormente tomar la decisión más adelante en el entrenamiento. Además, los pesos y el flujo tras cada capa lleva arrastrando todo el contexto anterior, no hay un límite en cuanto a la "cantidad" de contexto que se puede llevar cargando. Al menos en el modelo base de RNN.

El entrenamiento de una RNN es un poco distinto al de un MLP tradicional. El back-propagation de las recurrentes sucede en dos pasos: primero se realiza inferencia hacia adelante, calculando tanto  $h_t$  como  $y_t$ , y se acumula la pérdida tras cada avance a lo largo del tiempo, siempre guardando los valores de las capas ocultas tras cada paso para su uso

posterior. Después, el segundo paso, es el procesamiento de la secuencia en reversa, donde se calculan los gradientes requeridos, logrando el cómputo y guardado del término de error para usarlo en la capa oculta para cada paso hacia atrás. Jurafsky menciona que este proceso es llamado: "backpropagation a través del tiempo".

Los modelos de lenguaje de las RNN procesan las secuencias de entrada una palabra a la vez. Es decir, lo que se intenta es predecir la palabra que sigue a partir de la palabra actual inmediata y el estado oculto anterior. Es por ello que las RNN se diferencian claramente de los n-gramas, pues estos tienen contexto limitado, y a diferencia del contexto fijo de modelos clásicos, son un sistema que no tiene límite de contexto en cuanto a todo lo que puede arrastrar desde sus secuencias de entrada iniciales.

## Arquitectura Utilizada e Hiperparámetros

Finalmente, para establecer una línea base de comparación, se implementó un modelo utilizando una Red Neuronal Recurrente simple (RNN). Aunque es susceptible al problema del desvanecimiento del gradiente en secuencias muy largas, su arquitectura más sencilla sirve como un punto de referencia fundamental para evaluar la ganancia en rendimiento obtenida con las arquitecturas más complejas de compuertas como GRU y LSTM.

Al igual que con los modelos anteriores, la configuración se mantuvo consistente para los entrenamientos a nivel de carácter y palabra.

### Estructura del Modelo

La arquitectura del modelo RNN se diseñó con una mayor dimensionalidad en su estado oculto para compensar la simplicidad relativa de su unidad recurrente. La estructura fue la siguiente:

1. **Capa de Embedding:** Una capa inicial para convertir los tokens de entrada en vectores densos. Se asumió una dimensión de embedding de 128 para mantener la consistencia con las otras arquitecturas.
2. **Capas RNN Apiladas:** El núcleo del modelo consiste en 4 capas de RNN simple apiladas. Cada capa fue configurada con una dimensión de estado oculto (`hidden_dim`) de 512 unidades, un tamaño mayor para aumentar la capacidad de representación del modelo.
3. **Capa de Regularización (Dropout):** Se aplicó una tasa de Dropout del 30 % (0.3) entre las capas recurrentes para controlar el sobreajuste.

4. **Capa de Salida (Clasificador):** Una capa lineal final proyecta la salida de la última capa RNN al tamaño del vocabulario, utilizando una función de activación Softmax para la predicción del siguiente token.

### Hiperparámetros de Entrenamiento

Los hiperparámetros utilizados para el entrenamiento del modelo RNN se detallan en la Tabla 2. Se optó por un número menor de épocas de entrenamiento en comparación con los otros modelos, debido a la mayor rapidez con la que las RNN simples tienden a converger o a sobreajustar.

Cuadro 2: Resumen de hiperparámetros de entrenamiento para el modelo RNN simple.

| Hiperparámetro                      | Valor Utilizado    |
|-------------------------------------|--------------------|
| Optimizador                         | Adam               |
| Tasa de Aprendizaje (Learning Rate) | 5e-4 (0.0005)      |
| Función de Pérdida                  | Cross-Entropy Loss |
| Tamaño del Lote (Batch Size)        | 32                 |
| Épocas de Entrenamiento             | 12                 |
| Semilla Aleatoria (Seed)            | 42                 |

### Resultados

```
I <UNK> <UNK> <UNK> <UNK> <UNK> <UNK> <UNK> p l a n n e d   T h e r e ' s
s o m e o n e e l s e ' s d r e a m s ? ( E l s e ' s d r e a m s ?
( E l s e ' s d r e a m s ? W h i n e a l l , k n o w T h a t t h
e y t o l d m e I w a s g o n e B u t I ' l l n e v e r b e t
t e r o f f m y h e a d ) L e a r n i n g d a r d y s o h a p p e
n ? W h a t h a v e a c h a i n ' s i c s i n y o u r p a c e s
, B l u r r y f a c e s o f f e e l i n ' s p u l l s t i l l a r
e s i t c i n g G o t t o t h i s t i m e I ' l l s i t h e r e
' t i l I f i n d t h e p r o b l e m N o , I m o v e s l o w
I w a n t t o s t o p t i m e I ' l l s i t h e r e ' t i l I
f i n d t h e p r o b l e m N o , I
```

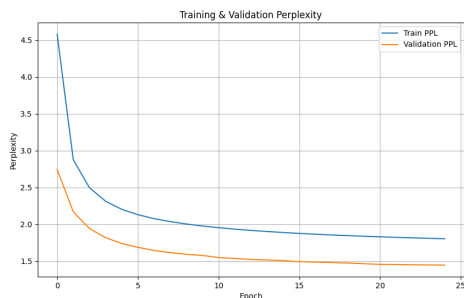
Listing 1: Output crudo del modelo generativo a nivel de carácter.

```
<UNK> like my soul but not my mind is n't helping , saturday when it 's for
the night 's decor it reveals what we dream of i have to know and i 'm
a lot , right in one piece now i asked me , " son , when i grow old will
you buy me a house of gold ? and when your father turns to stone will
you take care of me ? " i will make you queen of everything you see i '
ll put you on the map i 'm driving ' our thumbs for ammunition i was
born a choker nobody 's comin ' for me comin ' for me from it 's decidin
fine that part they form a lot with it 's real 'cause i told you you
are out of my mind heard you say ( " today " 'cause the heat , place
chances straight ever smell , you know that kills 's coming and welcome
to the style you have n't seen in a while it 's lavish ( feeling gourmet
) welcome to the new way of living it 's just the beginning of lavish
from the floor ( we 're broken too , yeah )
```

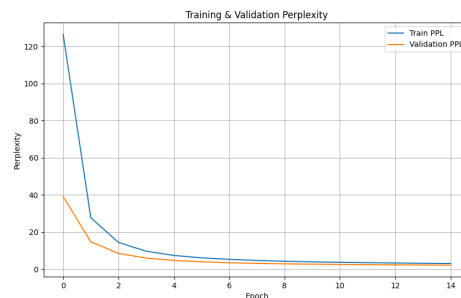
Listing 2: Output crudo del modelo generativo a nivel de palabra.

## PPL por época

Figura 3: Comparación de la perplejidad (PPL) por época para los modelos RNN.



(a) Modelo a Nivel de Carácter.



(b) Modelo a Nivel de Palabra.

## Conclusiones

En primer lugar, podemos mencionar que los resultados de PPL muestran una clara diferencia entre los modelos a nivel de carácter y a nivel de palabra. El modelo char-level alcanzó valores de validación cercanos a 1.44 tras 25 épocas, indicando una buena capacidad de predicción local, aunque con salidas generativas afectadas por la presencia de



múltiples <UNK>. Esto es muy probablemente debido a la forma en que se introdujo el prompt inicial, pues no se tokenizó a nivel carácter por carácter, lo que impidió al modelo explotar de manera correcta su estado inicial. A pesar de ello, la disminución progresiva y estable de la PPL parece sugerir que el modelo aprendió bien acerca de regularidades ortográficas y patrones.

Por otra parte, el modelo word-level presentó inicialmente una perplejidad muy alta, de hecho, superior a 100, pero convergió rápido hasta valores cercanos a 2.08 en el conjunto de validación. Esto refleja que, aun con un corpus de tamaño limitado, las representaciones distribuidas de palabras permiten capturar dependencias más semánticas y temáticas. Esta muestra refleja que a nivel palabras hay una mejor coherencia y frases reconocibles dentro del estilo lírico de Twenty One Pilots, aunque no sin redundancias y repeticiones frecuentes.

La métrica de novelty confirma dichas tendencias. Los modelos LSTM y GRU (que veremos más adelante), a nivel palabra, alcanzaron los valores más altos de novedad trigramal con  $\sim 0.92$ , mientras que la RNN simple mostró mayor tendencia a memorizar, con un valor de 0.22 en trigramas. Esto implica que las arquitecturas con puertas de memoria favorecen una mejor capacidad de recombinar fragmentos de texto en lugar de limitarse a repetir secuencias vistas durante el entrenamiento.

En cuanto a la calidad cualitativa de las letras generadas, los outputs a nivel carácter tienden a generar secuencias ruidosas y poco interpretables, mientras que a nivel palabra están mejor estructurados, con una mejor gramática, mejores referencias líricas y elementos estilísticos asociados a la banda, aunque noto que esto provoca la repetición de frases que vuelve al modelo poco creativo. Esto es reflejo de las limitaciones por el tamaño del corpus, provocando poca diversidad y la generación de clichés.

Adelantándonos un poco al siguiente modelo, la RNN presentan mayor tendencia a memorizar y a repetir. Asimismo, la generación a nivel palabra logra capturar mejor el estilo y temática del corpus. Ojalá tuviéramos mil canciones de Twenty One Pilots más, para poder entrenar un LM más sólido.

## Parte A | LSTM

Entrena los modelos con las canciunes y con las arquitecturas RNN/LSTM/GRU.  
A nivel caracter y a nivel palabra.

Uno de los principales problemas de las RNN es justo lo que volvió a aquella arquitectura una herramienta tan prometedora en los inicios del NLP como lo conocemos ahora: su contexto. Recordemos que las RNN arrastran el contexto de las capas de inicio, sin importar qué tan a profundidad vaya el entrenamiento.

De acuerdo con Dan Jurafsky, entre las razones por las cuales las RNN no logran capturar, seleccionar y llevar adelante la información más importante es que se le solicita a los pesos y a las capas ocultas que "proporcionen información útil para la decisión actual, y actualizar y llevar hacia adelante la información requerida para decisiones futuras".

Además, justo ir cargando con toda la información previa, y realizar la retropropagación a lo largo del tiempo, sumando la pérdida de cada capa, paso a paso, provoca lo que se conoce como "desvanecimiento del gradiente".

La solución a este tipo de problemas acarreados por las RNN llegó tras el diseño de arquitecturas que lograban no solo mantener contexto relevante a lo largo del tiempo, sino también lograr olvidar la información no necesaria, y recordar también la información requerida para el futuro.

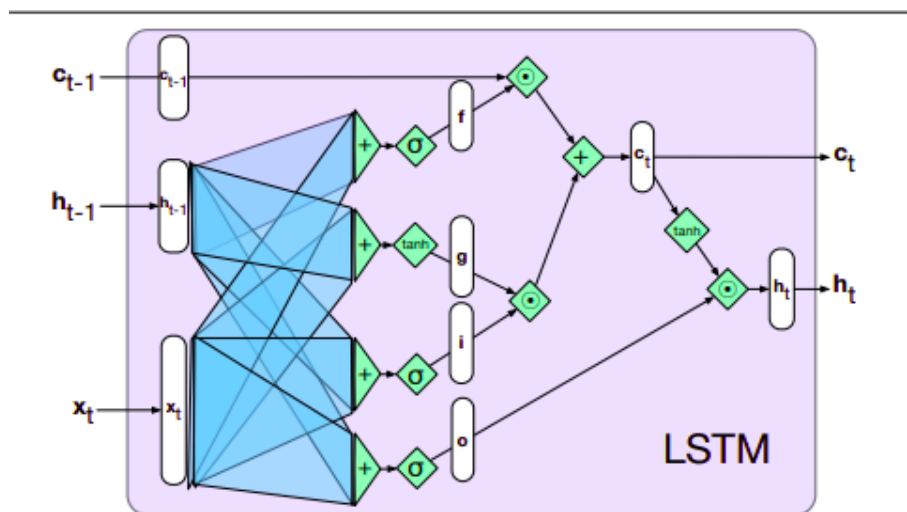


Figura 4: Arquitectura de una LSTM | Tomada de 'Speech Recognition' de Dan Jurafsky.

Es así como se llegó a la creación de la Long Short-Term Memory (LSTM) o Red de Memoria a Largo y Corto Plazo. Este diseño no es nuevo, surgió en 1997, pero como muchas de las arquitecturas de Redes Neuronales, no fue sino hasta la década pasada que contamos con recursos suficientemente poderosos para poder entrenarlas y ponerlas en producción.

Las LSTM realizan dos tareas principales: por un lado se encargan de eliminar información que ya no se necesita para el contexto, y por el otro lado capturan información que probablemente se utilice para la decisión siguiente. Este tipo de modelo logra aquello haciendo uso de una capa de contexto dentro de la arquitectura, además de las capas ocultas recurrentes propuestas en las RNN. Así, y mediante el uso de neuronas que utilizan compuertas para controlar el flujo de la información, logran hacer la captura del contexto relevante y no relevante.

Dan Jurafsky lo explica de la siguiente manera: "las compuertas de una LSTM comparten un patrón de diseño común: cada una consiste en una capa de propagación hacia adelante, seguida de una función de activación sigmoide, seguida de una multiplicación elemento a elemento con la capa que se está controlando". Recordemos que la sigmoide es una función que "empuja" las salidas hacia el 0 o hacia el 1, de tal modo que al combinarla con multiplicaciones elemento a elemento, lo que estaremos logrando es poner en funcionamiento una máscara binaria que decide si los valores aprendidos en ese momento son eliminados o no.

## Compuerta del Olvido (forget gate)

Esta compuerta tiene como propósito eliminar información del contexto que ya es útil para la toma de decisiones futura. El proceso de eliminación es sencillo, se realiza una suma ponderada de la capa oculta del estado anterior y la entrada actual, para posteriormente pasarlo a una Sigmoide. Posteriormente, esta máscara binaria se multiplica elemento a elemento por el vector de contexto para eliminar la información que ya no es necesaria. Se define la multiplicación elemento a elemento como  $\odot$ , a este producto también le llamamos "producto de Hadamard".

$$f_t = \sigma(U_f h_{t-1} + W_f x_t)$$

$$k_t = c_{t-1} \odot f_t$$

Ahora bien, para calcular la información que se va a extraer del estado oculto previo, y las entradas actuales, se utilizac:

$$g_t = \tanh(U_g h_{t-1} + W_g x_t)$$

## Compuerta de Adición (add gate)

Para seleccionar la información que se agregará al contexto actual se utiliza la puerta de adición. Esta se define de la siguiente manera:

$$i_t = \sigma(U_i h_{t-1} + W_i x_t)$$

$$j_t = g_t \odot i_t$$

Y posteriormente sumamos esto al vector de contexto modificado para obtener nuestro nuevo vector de contexto:

$$c_t = j_t + k_t$$

La compuerta de adición filtra la información nueva que está por ser añadida desde  $g_t$ . La diferencia clave entre la compuerta de adición y del olvido es sobre qué vector se aplican: la primera se aplica a la información nueva y la segunda se aplica al contexto previo.

## Compuerta de Salida (output gate)

La última compuerta en una LSTM base es la compuerta de salida. En ella se decide qué información se requiere para que el estado oculto actual. Es decir, este filtro lo que quiere hacer es tomar la decisión de que datos nos sirven para la decisión presente del modelo, no la que debe ser preservada para el futuro. Nuevamente, su estructura es similar a las anteriores:

$$o_t = \sigma(U_o h_{t-1} + W_o x_t)$$

$$h_t = o_t \odot \tanh(c_t)$$

## Arquitectura Utilizada e Hiperparámetros

De manera análoga al modelo GRU, se implementó y entrenó una arquitectura basada en redes de Memoria a Corto y Largo Plazo (LSTM, por sus siglas en inglés). La elección de LSTM permite una comparación directa con la arquitectura GRU, ya que ambas están diseñadas para mitigar el problema del desvanecimiento del gradiente en secuencias largas, aunque lo logran con mecanismos de compuertas (gates) ligeramente diferentes.

Nuevamente, la arquitectura se mantuvo idéntica para los entrenamientos a nivel de carácter y de palabra, con el fin de realizar una evaluación consistente.

### Estructura del Modelo

El modelo LSTM se construyó con la siguiente configuración de capas:

1. **Capa de Embedding:** Una capa inicial para mapear los tokens de entrada a vectores densos de 128 dimensiones.
2. **Capas LSTM Apiladas:** El núcleo del modelo está compuesto por una pila de 6 capas LSTM, una configuración más profunda que la del modelo GRU para explorar la capacidad del modelo de aprender jerarquías de características más complejas. Cada capa cuenta con un estado oculto de 256 unidades.
3. **Capa de Regularización (Dropout):** Se aplicó una tasa de Dropout del 30 % (0.3) entre las capas LSTM para prevenir el sobreajuste, una medida especialmente importante en arquitecturas más profundas.
4. **Capa de Salida (Clasificador):** Una capa lineal final transforma la salida de la última capa LSTM al tamaño del vocabulario, empleando una activación Softmax para generar la distribución de probabilidad del siguiente token.

### Hiperparámetros de Entrenamiento

Los hiperparámetros de entrenamiento para el modelo LSTM se definieron como se muestra en la Tabla 3. Notablemente, se utilizó un tamaño de lote mayor y un número superior de épocas en comparación con el modelo GRU para evaluar el comportamiento de la red bajo condiciones de entrenamiento más extensivas.

Cuadro 3: Resumen de hiperparámetros de entrenamiento para el modelo LSTM.

| Hiperparámetro                      | Valor Utilizado    |
|-------------------------------------|--------------------|
| Optimizador                         | Adam               |
| Tasa de Aprendizaje (Learning Rate) | 5e-4 (0.0005)      |
| Función de Pérdida                  | Cross-Entropy Loss |
| Tamaño del Lote (Batch Size)        | 64                 |
| Épocas de Entrenamiento             | 15                 |

### PPL por época

### Resultados

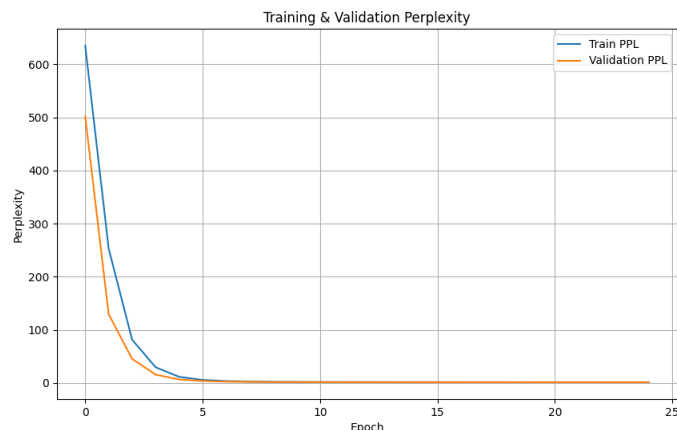


Figura 5: Modelo a Nivel de Palabra.

```
I <UNK> <UNK> <UNK> <UNK> <UNK> <UNK> <UNK> s way that it shou
ld to let this reat I don't care what's
in your hair I just wanna know what's on
your mind I used to say I wanna die befo
re I'm old But because of you, I might th
ink twice Yeah, yeah, yeah! Yeah, yeah,
yeah! Oh-oh-oh-oh-oh Yeah, yeah, yeah
! Oh-oh-oh-oh-oh Yeah, yeah, yeah! Oh-o
h-oh-oh-oh Yeah, yeah, yeah! Oh-oh-oh-
oh-oh Yeah, yeah, yeah! Oh-oh-oh-oh-oh
Yeah, yeah, yeah! Oh-oh-oh-
```

Listing 3: Output crudo del modelo LSTM a nivel de carácter.

```
<UNK> like my soul but not my mind and you're living they recorded that up
it) slowin' do i'm goin' all i feel some wonderin' would you be ( know
you be? ) my soul 'cause is the only night, i know waging my side and
the floor how my face some doors take i'm only yeah ( hates that she
wrote he made in a binger and he live out if your passing gone it's
another list and said my dream i'm pink, a car locomotive, and i didn't
take it oh-oh-oh, they'r? asking for a second try you and i will never
say it no, no, is not a little that's come up with start they're way
same who-o-oa so us you've met but i am dying a friend, (ooh) home can
won't you say that of a ride inspires you are not in a dream? neon
```

```
waitin', wait, someone one oh, <EOS>
```

Listing 4: Output crudo del modelo LSTM a nivel de palabra.

## Conclusiones

El modelo LSTM, entrenado con un mayor número de épocas y un tamaño de lote superior al de GRU, mostró un desempeño robusto en términos de perplejidad. A nivel caracter, la PPL de validación descendió de 7.75 en la primera época a valores cercanos a 1.15 en la época 12, mostrando una notable capacidad de modelar dependencias locales en secuencias de símbolos. El output generado, aunque afectado por la presencia de tokens <UNK> de inicio, alcanzó una estructura fonética y rítmica más clara, incluyendo patrones de repetición como “yeah, yeah, yeah”. Si bien estos patrones son redundantes, reflejan rasgos estilísticos que no se presentaron en, por ejemplo, el modelo de RNN.

A nivel palabra, la PPL inicial fue muy alta, superior a los 600, pero decreció de manera constante hasta valores de validación cercanos a 2.24. Esto indica que el modelo aprendió a asignar probabilidades cada vez más concentradas a secuencias posibles, aproximándose a un nivel de producción muy bueno. El texto de output contiene frases de coherencia y con referencias reconocibles dentro del imaginario de la banda. Sin embargo, aún presenta estructuras fragmentadas y saltos semánticos abruptos.

En cuanto a las métricas de novelty, el modelo LSTM alcanzó un equilibrio interesante. La proporción de novedad en trigramas fue de 0.92, muy cercana a la de la GRU, y superior la de la RNN. Esto nos demuestra que, además de mantener memoria de secuencias frecuentes, la LSTM pudo recombinar fragmentos de manera creativa, evitando caer en mucha repetición. A nivel bigramas, se observó una tendencia ligeramente mayor hacia la novedad con 0.65 puntos que confirman la capacidad de explorar combinaciones nuevas sin perder del todo la coherencia local.

A nivel caracter, la PPL de este modelo es bastante bajo. A nivel palabra, la convergencia es lenta, pero se sostiene al lograr niveles muy aceptables de PPL confirmando la utilidad del LSTM para capturar tanto la semántica como la variedad léxica dentro de un corpus pequeño. Este modelo supera a la RNN en cuanto a la reducción de la memorización y la maximización de la novelty. Sus resultados son comparables con los de la GRU, con ligera ventaja en coherencia global.

## Parte A | GRU

Entrena los modelos con las canciunes y con las arquitecturas RNN/LSTM/GRU. A nivel caracter y a nivel palabra.

En el contexto de las RNN, sabemos que estas se mueven sin una ventana fija. El estado oculto  $h_t$  resume el pasado y se actualiza con el nuevo  $x_t$ , es decir, la palabra en la posición  $t$ , para predecir la salida  $y_t$ . En una RNN clásica, el cálculo se organiza en tres bloques: de la capa de entrada al estado oculto, del estado oculto a otro estado oculto, y de ese estado oculto a la salida. El entrenamiento del modelo de lenguaje se hace por auto-supervisión con entropía cruzada como función objetivo, donde la probabilidad de la secuencia se factoriza paso a paso y se optimiza la pérdida.

Sin embargo, RNN sufre del desvanecimiento del gradiente. Agregar compuertas como hace la LSTM resuelve ese problema al descartar información y decidir la que sí es de importancia para las decisiones. Dentro de la familia de RNN con compuertas se encuentra también la GRU.

Conocida como Gated Recurrent Unit (GRU), este tipo de arquitecturas intenta mantener un desempeño competitivo con las LSTM a pesar de introducir menos parámetros.

### La célula GRU y sus compuertas

Una GRU controla el flujo de información con dos compuertas sigmoideas por dimensión: la compuerta de actualización  $z_t$  y la compuerta de reinicio llamada  $r_t$ . La idea clave frente a la LSTM es que una sola compuerta gobierna a la vez el "olvido" del pasado y la decisión de escribir el nuevo estado, simplificando la arquitectura.

Las ecuaciones de actualización son:

- **Compuerta de actualización:**  $z_t = \sigma(b_z + U_z x_t + W_z h_{t-1})$
- **Compuerta de reinicio:**  $r_t = \sigma(b_r + U_r x_t + W_r h_{t-1})$
- **Candidato de estado:**  $\hat{h}_t = \tanh(b + U x_t + W(r_t \odot h_{t-1}))$
- **Mezcla final:**  $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \hat{h}_t$

La intuición detrás de esto es que  $z_t$  implementa un integrador con fuga condicional. i.e. cerca de 1 "copia" el estado oculto  $h_{t-1}$ ; mientras que cerca de 0 "reescribe" con  $\hat{h}_t$ . Por otra



parte,  $r_t$  decide cuánto del pasado participa en el cómputo del candidato  $\hat{h}_t$ , introduciendo no linealidad adicional a la relación entre lo pasado y lo futuro.

A nivel estructural, la GRU mantiene los tres bloques U, W, V de una RNN, pero sustituye la transición oculta a oculta por una transición puerta a dependiente. Al compararse con una LSTM, la GRU elimina el estado de celda explícito y la salida está directamente en  $h_t$ . De esa manera, intenta ahorrar recursos volviendo a la GRU una LSTM "destilada". Ambos modelos suelen ser competitivos entre ellos, y su uso depende del problema específico a atender.

## Arquitectura Utilizada e Hiperparámetros

Para la tarea de generación de texto se implementó y evaluó un modelo basado en la arquitectura de Unidades Recurrentes Gated (GRU, por sus siglas en inglés). Esta arquitectura fue seleccionada por su eficiencia y su capacidad para capturar dependencias a largo plazo en secuencias, siendo una alternativa robusta a las redes LSTM.

El modelo fue entrenado tanto a nivel de carácter como a nivel de palabra para realizar una comparación exhaustiva. La arquitectura y los hiperparámetros se mantuvieron consistentes entre ambas modalidades para asegurar una evaluación justa de los resultados. A continuación, se detalla la configuración específica utilizada para los modelos GRU.

### Arquitectura del Modelo GRU

El diseño del modelo se construyó como una pila de capas secuenciales, utilizando el framework PyTorch. La estructura fue la siguiente:

1. **Capa de Embedding:** Una capa inicial que transforma los índices de los tokens (sean caracteres o palabras) en vectores densos. La dimensión de estos vectores de embedding se fijó en 128.
2. **Capas GRU Apiladas:** El núcleo del modelo consiste en 4 capas GRU apiladas, cada una con una dimensión de estado oculto (`hidden_dim`) de 256 unidades. El apilamiento de capas permite al modelo aprender representaciones de los datos a diferentes niveles de abstracción.
3. **Capa de Regularización (Dropout):** Para mitigar el sobreajuste, se aplicó una capa de Dropout entre cada una de las capas GRU, con una tasa de desactivación de neuronas del 30 % (0.3).

4. **Capa de Salida (Clasificador):** Finalmente, una capa lineal (densa) proyecta el estado oculto de la última capa GRU al tamaño del vocabulario, seguida de una función de activación Softmax para obtener una distribución de probabilidad sobre el siguiente token a predecir.

### Hiperparámetros de Entrenamiento

El proceso de entrenamiento fue configurado con los hiperparámetros que se resumen en la Tabla 4. La elección de estos valores se basó en una combinación de prácticas estándar y ajustes empíricos para la tarea.

Cuadro 4: Resumen de hiperparámetros de entrenamiento para el modelo GRU.

| Hiperparámetro                      | Valor Utilizado    |
|-------------------------------------|--------------------|
| Optimizador                         | Adam               |
| Tasa de Aprendizaje (Learning Rate) | 5e-4 (0.0005)      |
| Función de Pérdida                  | Cross-Entropy Loss |
| Tamaño del Lote (Batch Size)        | 32                 |
| Épocas de Entrenamiento             | 15                 |
| Semilla Aleatoria (Seed)            | 42                 |

### Resultados

```
I <UNK> <UNK> <UNK> <UNK> <UNK> <UNK> <UNK> v i s p e n s e a n d a b o
v e m y t h r o a t ? S h a d o w s w i l l s c r e a m t h a t I
' m a l o n e - l o n e - l o n e I - I - I - I ' v e g o t a m i
g r a i n e A n d m y p a i n w i l l r a n g e f r o m u p , d
o w n , a n d s i d e w a y s T h a n k G o d i t ' s F r i d a y
' c a u s e F r i d a y s w i l l a l w a y s b e o n a s o n g
h e w r o t e T h i s i s t h e l a s t t i m e , t h i s i s
t h e l a s t t i m e , t h i s i s t h e l a s t t i m e , t h
i s i s t h e l a s t t i m e , t h i s i s t h e l a s t t i m
e E n t e r t a i n m y f a i t h T h i s i s t h e l a s t t i
m e , t h i s i s t h e l a s t t i m e
```

Listing 5: Output crudo del modelo GRU a nivel de carácter.

```
<UNK> like my soul but not my mind have i have back when a beautiful day  
but you me, around they will never don't want to know you who i see do i  
will you'll don't take a piece, of gold? to someone tryin' all next  
like what i wanna your footing, of nothing but i am i'll keep movin' a  
last feeling that come in black it's now what i know, up they get cold,  
sitting i get down) i get it's they just get to be all you never have it  
again never know they morph of a kingdom, join did a concert, i'll  
decorates with instead is a good progression catch i don't know i had to  
do one on my pride lean on my pride, i'm a lion i should the count i  
know the gesture we be slow i sing a day for break with a little cheetah  
after a re-beginning moment's mending memories dormant the beeps and  
tones keep would that not be 'cause me, are frozen and the mainland is  
climbing flying but we can the picture (am then my shirtsleeve shadows  
jacket not your wits then you mine to subliminally get out start (ooh-  
ooh) i don't go through you don't
```

Listing 6: Output crudo del modelo GRU a nivel de palabra.

## Conclusiones

El modelo GRU mostró un comportamiento intermedio entre la simplicidad de la RNN y la capacidad de memoria extendida de la LSTM. El comportamiento de la PPL demostró que, para este modelo de GRU, a nivel carácter descendió rápidamente hasta estabilizarse en valores bajos, los cuales fueron comparables con los alcanzados por la LSTM. En cuanto al texto generado, mostró patrones fonéticos precisos y estructuras repetitivas con ritmo lírico, así como la toma de secuencias reconocibles de Twenty One Pilots como “*Shadows will scream that I'm alone*” o “*I've got a migraine*” de Holding On To You y de Migraine, ambas canciones muy populares de la banda. Esto demuestra que el modelo estaba un tanto limitado en cuanto a la creatividad, pero que lograba encontrar secuencias líricas de importancia.

Respecto a los resultados de la GRU a nivel palabra, la PPL mostró una disminución constante hasta valores relativamente bajos, aunque superiores a los alcanzados por la LSTM. Los outputs generados reflejaron cierta coherencia temática, con construcciones gramaticales más estables que las obtenidas por la RNN. Se observaron frases como “*i'll keep movin*” o “*they morph of a kingdom*”, que si bien tenían una semántica clara, no se conectaban del todo bien entre ellas. Esto refleja que nuestra GRU logra capturar contexto de medio alcance, que tiene dificultades con alcanzar la fluidez global de una LSTM.

Los resultados de novelty confirman lo anterior. La GRU obtuvo una novedad en triga-

mas de 0.92, la más alta en nuestros tres modelos de redes neuronales, pero en bigramas una de solo 0.62. Esto nos parece estar diciendo que la arquitectura es especialmente efectiva para evitar repetición de manera excesiva, y explorar combinaciones nuevas sin sacrificar del todo la coherencia. Al compararlo con la RNN, se mostró una mayor memorización, mientras que el LSTM demuestra un balance más equilibrado entre memoria novedad.

De ese modo, podemos decir que la GRU es un modelo que cae justo en medio de la simpleza de la RNN y sus problemas de memorización, y el entendimiento global y complejo de la LSTM. Es competitiva con la LSTM y supera claramente a la RNN. Quizás un modelo mejor regularizado, con un corpus más amplio y más tiempo de cómputo pueda demostrar ser una alternativa clara y eficaz a la LSTM.

## Introducción a los Transformers y su relación con los LLM

Los modelos de lenguaje han recorrido un largo camino desde los enfoques estadísticos basados en conteos de n-gramas hasta convertirse en los actuales modelos de lenguaje a gran escala, también conocidos como LLMs por sus siglas en inglés. En ese sentido, la arquitectura Transformer (Vaswani et al., 2017) ha sido el principal punto de partida, con el cual se revolucionó la manera en que presentamos y procesamos secuencias lingüísticas.

A diferencia de los modelos recurrentes, como las RNN, LSTM o GRU, que procesan el texto paso a paso, y que sufrían de limitaciones para capturar dependencias largas, los Transformers incorporaron un mecanismo esencial para su éxito: la auto-atención (o *Self-Attention*). Este mecanismo permite que cada palabra o token en una secuencia "se fije" de manera directa en cualquier otro, sin importar la distancia que los separe, construyendo representaciones contextuales mucho más ricas y globales.

Este avance fue toda una revolución para el Procesamiento de Lenguaje Natural, pues creó modelos que podían manejar contextos extensos y aprender patrones complejos de manera eficiente. Como resultado, los Transformers se convirtieron muy rápidamente en la arquitectura dominante en prácticamente todas las tareas del NLP, desde la traducción automática, hasta la generación de texto.

Los LLMs contemporáneos como GPT, LLaMA, Gemini o DeepSeek no son más que instancias a gran escala de esta arquitectura. Es decir, lo que se ha hecho es tomar la arquitectura y volverla gigantesca, de tal modo que ahora tenemos modelos sumamente eficaces. La característica de *Large* de los LLMs se refiere tanto a su número de parámetros como a la magnitud del corpus con el que fue entrenado. Este diseño, que no necesita de convoluciones ni de recurrencia, representa un cambio de paradigma en la modelación del lenguaje. Se trata de una arquitectura altamente paralelizable y que puede capturar dependencias globales en el texto de manera directa.

### El cambio de paradigma

El Transformer (Vaswani et al., 2017), es una arquitectura de red neuronal diseñada originalmente para tareas de traducción automática, pero que pronto se convirtió en el estándar para el NLP, encontrando aplicaciones en muchos campos.

A diferencia de los modelos recurrentes, que procesan de manera secuencial el texto, el Transformer elimina la recurrencia y se basa en bloques paralelos compuestos de:

- Mecanismo de self-attention o multi-head attention.
- Redes feed-forward posicionales.

- Normalización por capas y conexiones residuales.

En general, un Transformer puede adaptar tres variantes arquitectónicas principales:

- **Decoder-only**: utilizado en modelos generativos como GPT, LLaMA o Mistral.
- **Encoder-only**: especializado en clasificación y comprensión como BERT, Beto o RoBERTa.
- **Encoder-decoder**: comúnmente utilizados para traducción y tareas secuencia a secuencia.

Una de sus grandes ventajas es que pueden procesar contexto completo en paralelo, y aprender dependencias de largo alcance en un solo paso, como no podrían los modelos recurrentes.

## Self-Attention

Dado un conjunto de representaciones entrada  $(x_1, x_2, \dots, x_n) \in \mathbb{R}^d$ , una por cada token, la auto-atención produce nuevas representaciones  $a_1, a_2, \dots, a_n$  donde cada  $a_i$  es una combinación ponderada de todos los vectores  $x_j$  donde  $j < n$ .

La idea es que un token puede consultar información de otros tokens en función de su similitud semántica y contextual.

Se definen tres proyecciones aprendidas de cada vector de entrada:

$$q_i = x_i W^Q, \quad k_i = x_i W^K, \quad v_i = x_i W^V$$

Donde  $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$  son matrices de parámetros.

La similaridad entre token  $i$  (consulta) y otro token  $j$  (clave) se calcula mediante un producto punto escalado:

$$\text{score}(i, j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

El uso del factor  $\sqrt{d_k}$  estabiliza los gradientes y evita valores extremos al aplicar la exponenciación en el softmax. Estas puntuaciones se normalizan con una Softmax, obteniendo los pesos de atención:

$$\alpha_{ij} = \frac{\exp(\text{score}(i, j))}{\sum_{m=1}^n \exp(\text{score}(i, m))}$$

Finalmente, la representación contextual del token  $i$  se obtiene como una suma ponderada de los valores:

$$\alpha_i = \sum_{j=1}^n \alpha_{ij} v_j$$

Este mecanismo se aplica en paralelo para todos los tokens de la secuencia, generando la matriz de salidas  $A \in \mathbb{R}^{n \times d_v}$ .

### ***Multi-head Attention***

En la práctica, no se utiliza una sola cabeza de atención, sino múltiples. Con  $h$  cabezas, se calculan diferentes subespacios de atención:

$$\text{head}_h = \text{Attention}(XW_h^Q, XW_h^K, XW_h^V)$$

y luego se concatenan todas las salidas para formar la representación final:

$$\text{MHA}(X) = [\text{head}_1; \dots; \text{head}_h]W^O$$

donde  $W^O$  es una matriz de proyección adicional.

## **Transformers populares**

La introducción de la arquitectura Transformer no sólo transformó la investigación en PLN, sino que también dio origen a una serie de modelos que hoy son considerados referentes en la evolución de los modelos de lenguaje a gran escala (**LLMs**). Entre ellos destacan las familias **GPT**, **LLaMA** y más recientemente **DeepSeek**, cada una con enfoques y motivaciones distintas.

### **GPT (Generative Pretrained Transformer)**

Los modelos GPT, desarrollados por OpenAI, constituyen el ejemplo paradigmático de los Transformers decoder-only. Su entrenamiento se basa en la tarea de *next-word prediction*, es decir, la predicción autoregresiva de la siguiente palabra dada una secuencia de entrada.

- **GPT-1 (2018)** introdujo la idea de preentrenar un gran modelo en corpus generales y luego ajustarlo (*fine-tuning*) para tareas específicas.

- **GPT-2 (2019)** escaló el tamaño (1.5B parámetros) y mostró por primera vez capacidades generativas sorprendentes, aunque inicialmente fue liberado de forma parcial por preocupaciones de seguridad.
- **GPT-3 (2020)**, con 175B parámetros, consolidó el paradigma de los LLMs al mostrar *emergent abilities*: razonamiento básico, traducción, generación de código y diálogo. Su ajuste posterior dio origen a ChatGPT, que incorporó *instruction tuning* y *reinforcement learning from human feedback* (RLHF) para mejorar su alineación con usuarios.

El impacto de GPT fue doble: estableció la viabilidad de los LLMs en múltiples dominios y mostró que un único modelo, entrenado con suficiente escala, puede generalizar a una amplia variedad de tareas lingüísticas sin rediseñar la arquitectura.

### LLaMA (Large Language Model Meta AI)

En 2023, Meta presentó la familia LLaMA, concebida como un esfuerzo por abrir a la comunidad modelos competitivos pero con un enfoque en eficiencia y accesibilidad.

- **LLaMA-1** ofrecía variantes de 7B a 65B parámetros entrenadas con datos cuidadosamente filtrados.
- **LLaMA-2 (2023)** introdujo mejoras en el preentrenamiento y versiones afinadas para diálogo.
- **LLaMA-3 (2024)** alcanzó mayores tamaños (hasta 405B parámetros en versiones privadas), con una fuerte orientación a la investigación abierta y la compatibilidad con entornos de *fine-tuning*.

Una de las principales contribuciones de la familia LLaMA fue democratizar el acceso a modelos de calidad comparable a GPT-3, pero bajo licencias abiertas, favoreciendo la investigación académica y el desarrollo de aplicaciones personalizadas.

### DeepSeek

En el ecosistema más reciente ha surgido DeepSeek, desarrollado en China como un competidor directo de modelos occidentales de gran escala. Su relevancia radica en dos aspectos:

- **Optimización computacional:** uso de técnicas avanzadas de entrenamiento distribuido que permiten reducir costos energéticos y de hardware.



- **Competencia global:** representa un movimiento estratégico en la carrera internacional de LLMs, mostrando que la arquitectura Transformer puede adaptarse a entornos con recursos limitados pero ambiciones de despliegue masivo.

DeepSeek no sólo apunta a competir en calidad, sino también a abrir el camino a modelos más eficientes, lo que sugiere que el futuro de los LLMs no depende únicamente de escalar parámetros, sino de optimizar su entrenamiento y uso.

### Otros modelos influyentes

Aunque GPT, LLaMA y DeepSeek ocupan hoy el centro de atención, existen otras familias de Transformers fundamentales en la evolución del campo:

- **BERT (2018):** modelo *encoder-only* basado en *masked language modeling*, diseñado para tareas de comprensión como clasificación, búsqueda semántica y análisis de sentimiento.
- **T5 (2019):** modelo *encoder-decoder* que formuló todas las tareas de PLN como problemas de traducción de texto a texto.
- **Mistral (2023):** modelos ligeros de arquitectura *decoder-only*, altamente eficientes, diseñados para ser competitivos con menos parámetros.
- **Gemini (2023, Google DeepMind):** sucesor de PaLM, con capacidades multimodales (texto e imagen) y mayor integración con entornos de búsqueda.

## Parte A | TinyLLaMA-1.1B

Afinar al menos un modelo Transformer en cada parte (generación y clasificación), eligiéndolo de Hugging Face.

### Resultados

```
I will take my heart with my hands and I'll lead you to believe that I am
  hiding something You don't need to run Heads are moving up and down, you
    got it It looks so much warmer there" <|startsong|> We used to play
pretend, give each other different names I said I fly by the dangerous
bend symbol (Wait, what? Wait, what?) And now they're outside, ready to
bust (And guess what?) Lean on my pride, I'm a lion Pretending to
understand how you think your world is ending (Keeps comin' around,
keeps comin', keeps comin' ) I found my way, right time, wrong place But
  now you double as a paper maker <|endsong|> Mm, ba-da-dum, ba-da-dum-
bum Dry us in the sand (Mm-mm) Da da, da da da, da-da) And when your
father turns to stone <|endsong|> (Hey, hey, hey, hey, hey, hey,
hey) Just keep it outside, keep it outside, yeah Let me know when you've
  had enough Quarantine Ooh, ooh You should go your way, I should try
mine Yeah
```

Listing 7: Output crudo del modelo TinyLLaMA-1.1B a nivel de palabra.

```
If you ever wanna see me just let me know We get bodies every day (Ooh-ooh,
  ooh-ooh) Just don't believe the hype I don't have a chance to say
  goodbye I want to stop time Pardon my delay Don't make me waste my love
  Tell our dad I'm sorry I check the clock, wondering what he'll pull this
  time I've got two faces, Blurry's the one I'm not I've been thinkin'
  too much He knows that it's almost over <|endsong|> I know, I know-ow-ow
  And now they're outside, ready to bust (And guess what?) I could take
  the high road (Eh, eh, eh, eh) I prayed those lights would take me home
  They walked up to me and when they got close You'll be in and out, out,
  out, man They asked me, "Son, when I grow old Ooh, ooh So move out of
  our way, we're pushing sideways Yeah, yeah, yeah! Take that Know I'll
  keep movin', know I'll keep movin' Entertain my, entertain my faith But
  I know that I'm goin' low Low-key
```

Listing 8: Output crudo del modelo TinyLLaMA-1.1B a nivel de palabra.

```
Dema will fall like the other guys (I'll fake you out) I want to be afraid,
  but it seems that these days It's a backslide But they say a system's
  coming in In time, I will leave the city My friends and I have problems
  If I keep moving, they won't know You don't need to run But I can hear
  my way around Down there, it's some shelter from the weather They think
  this thing is a highway, highway I wanna see it all, no surprises, yeah
  Just so I don't fall through the floor So high, so high We used to play
  pretend, give each other different names Yeah, yeah, yeah! Don't forget
  abou-bou-bou-bou-bout me And now I just sit in silence To scrape the
  frosted windshield like we're barely scrapin' by Push on through Ah-ah-
  ah-ah-ah-ah, ah-ah-ah-ah-ah-ah, ah-ah-ah-ah-ah-ah (ah-ah-ah-ah-ah-ah)
  Can you save? Can you save my? Save my Are you doin' good? No, not me,
  it's for a friend (Ooh) Do-do-do
```

Listing 9: Output crudo del modelo TinyLLaMA-1.1B a nivel de palabra.

## Conclusiones

El modelo TinyLLaMA-1.1B, a diferencia de las arquitecturas recurrentes tradicionales, mostró un desempeño significativamente superior en términos de perplejidad y calidad de los textos generados. Tras un proceso de *fine-tuning* ligero, con cuatro épocas y un *batch size* de dos unidades, la pérdida de entrenamiento descendió hasta valores cercanos a 0.61, lo cual corresponde con una PPL de aproximadamente 1.84. En el conjunto de validación,

la PPL se estabilizó en torno a un valor de 3.06, un valor muy bajo considerando la naturaleza del corpus. Estos resultados superan claramente lo obtenido por las arquitecturas anteriores (RNN, LSTM y GRU), especialmente a nivel de estabilidad y convergencia.

En cuanto a la generación de letras, TinyLLaMA fue capaz de articular secuencias más largas y coherentes, manteniendo tanto la estructura formal de las canciones como ciertos *motifs* característicos de Twenty One Pilots. Incrustó también conceptos que habitan el imaginario de la banda, como lo es *Blurryface*. A diferencia de las RNN, que tendieron a memorizar secuencias cortas, o las GRU que mostraban fragmentación, TinyLLaMA logró mantener consistencia global incluso a lo largo de estrofas enteras.

Sin embargo, el modelo también tiene limitaciones evidentes. Algunas secuencias derivaron en construcciones fragmentadas o clichés genéricos como: “*I found my way, right time, wrong place*”, una sentencia importante y reconocida de la canción The Judge. Esto refleja que, a pesar de que el modelo aprovecha su capacidad de atención, sigue condicionado al tamaño relativamente pequeño del corpus. La creatividad con la que se escribieron las canciones sigue siendo un tanto limitada, con muchas repeticiones que alteran la fluidez semántica por patrones rítmicos.

En suma, podemos afirmar que TinyLLaMA-1.1B representa un salto claro en calidad frente a las arquitecturas recurrentes. Su capacidad de auto-atención le permite modelar dependencias de largo alcance, manteniendo tanto la coherencia semántica como los patrones rítmicos propios de las letras. Aunque aún presenta clichés y algunas secuencias inconexas, la combinación de baja PPL, coherencia global y estilo lírico consistente lo posicionan como un modelo superior. Quizás un corpus más extenso y un ajuste más fino pueda superar ampliamente a esta implementación.

Hablando en términos más coloquiales, es claro que incluso un Transformer básico y pequeño supera a las arquitecturas recurrentes, demostrando cómo es que este paradigma cambió la manera en que se hace el NLP de alto nivel.

## Parte A | Mistral-7B LoRA

Afinar al menos un modelo Transformer en cada parte (generación y clasificación), eligiéndolo de Hugging Face.

### Mistral-7B LoRA

Al ser este el modelo con mejores resultados, creí adecuado colocar un apartado más técnico respecto a este transformer.

Mistral mantiene la arquitectura Transformer Decoder estándar, basada en *Self-Attention* y *Feed-Forward Networks*, pero introduce modificaciones que reducen complejidad y aumentan la eficiencia sin pérdida de capa expresiva.

Recordemos que cada bloque de atención se define como:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k} + M}\right)V$$

Donde  $M$  es la máscara causal que impide que el modelo “mire a futuro”. Sin embargo, mistral modifica el mecanismo con dos adiciones:

- **Sliding Window Attention (SWA):** en lugar de atender a todos los tokens previos, cada token solo atiende a los últimos  $W$  tokens. Por defecto esto es  $W = 4096$ . Esto reduce la complejidad de  $O(n^2)$  a  $O(nW)$ , permitiendo manejar contextos extensos con menor saturación de memoria.
- **Grouped-Query Attention (GQA):** esto reduce el número de cabezas de clave. En lugar de una proyección  $(Q, K, V)$  por cabeza, agrupa varias cabezas de query y  $h_{kv}$  las de key; entonces  $h_q < h_{kv}$ . Esto reduce la memoria y el tiempo de inferencia sin sacrificar la expresividad.

Además, cada bloque Transformer incluye una red feed-forward con activación SiLU (Swish):

$$\text{FFN}(x) = W_2 \text{SiLU}(W_1 x + b_1) + b_2$$

Con dimension expansion factor de  $4\times$  el tamaño del embedding (similar a LLaMA). La función de activación SiLU (Sigmoid Linear Unit), conocida también como Swish, es útil para Mistral porque combina lo mejor de ReLU con la Sigmoid, dando resultado a

un entrenamiento más estable y un mejor rendimiento general del modelo. Esta se define matemáticamente como:

$$\text{SiLU}(x) = x \cdot \sigma(x) = x \cdot \frac{1}{1 + e^{-x}}$$

A diferencia de ReLU, que tiene una esquina afilada con  $x = 0$ , SiLU es una curva suave y continua. Esta suavidad le permite al proceso de optimización ser más estable durante el entrenamiento. De igual forma se evitan cambios bruscos en los gradientes, permitiendo que el modelo converja de manera más eficiente. Además, para valores negativos pequeños, la función decrece antes de volver a cero.

Asimismo, Mistral-7B LoRA cuenta con un entrenamiento previo sobre 7.3B de parámetros, y 32 cabezas de atención. Como optimizador utiliza AdamW + cosine decay.

Ahora bien, hablando de LoRA (Low-Rank Adaptation), este mecanismo permite ajustar solo una pequeña parte de los parámetros del modelo sin modificar los pesos originales. La idea es que se descomponga una actualización de  $\delta W$  en cada matriz de pesos, transformándose en dos matrices de rango bajo:

$$W' = W + \delta W = W + BA \quad A \in \mathbb{R}^{r \times d}, B \in \mathbb{R}^{d \times r}$$

Donde el rango  $r$  es pequeño, para nuestro caso específico fue  $r = 16$ , lo que implica entrenar menos del 1 % de los parámetros totales. Esto reduce drásticamente el consumo de VRAM y permite ejecutar el fine-tuning completo en una sola GPU Titan RTX de 24 GB, que es lo que teníamos disponible en el servidor de CIMAT Lab.

## Resultados

```
In the city lights I find myself  
I don't know why  
I wanna stay inside tonight  
I think it's right  
But my heart keeps telling me that I should go  
Someplace I know where I belong  
So, I take a deep breath, step outside  
And follow my heart to where I want to hide  
  
In the shadows of my memories  
There's things I can't forget  
The times when you and I used to be  
Together we had what we wanted yet  
A sparkle in our eyes would leave us  
To chase dreams that only took us far from here  
Now, all those years seem like a blur  
And time has passed by, no trace left behind  
Like footprints in the sand  
My past steps slowly wash away  
I see them sinking with the tide  
Leaving me with a future I might not know  
  
If you could read my mind  
What would you think of who I am?  
Have I changed since then?  
Did your love bring out the best or the worst in me?  
Do you remember how I felt?
```

Listing 10: Output crudo del modelo Mistral-7B LoRA a nivel de palabra.

```
They say I'm broken but I'm breathing,  
I say I'll heal but don't know what I mean  
You see they say the heart never forgets,  
The pain that you feel just gets passed down to me  
And now it's like a game of catch,  
Every time I move, the feeling gets passed along  
  
Now we all talk about love and compassion,  
But how can we show it when there's nothing left?  
When every day feels more desperate than the last,  
It's easy to believe what they say  
  
They say, "Jena, there's something wrong with you,  
Don't you get it yet? You're already gone,"  
So I go looking for answers,  
Looking for a cure, anything to be strong  
  
"Jena, you're too broken, you won't get well,"  
They say, "Your mind is lost and there's no way back"  
  
Am I too far gone? Did I miss my chance?  
Do you think maybe there's hope left?
```

Listing 11: Output crudo del modelo Mistral-7B LoRA a nivel de palabra.



```
Sometimes my shadow sings louder than I do
A symphony of noise so loud it rings and echoes in the dark of night
It shines with light in its heart, and a deep soulful wisdom
The shadow was a creature of its own right
Sometimes we are just shadows of ourselves
Drowning in our own light as the real us watches on in the background
Someone else.
Some days I wake up and forget that I am me
And everything is a haze and then suddenly you're there
You bring back clarity

My best friend once told me that every person needs their best friend to
    see them for who they truly are
Every day I watch you and you help me to remember myself
When I look at your face I know the depths of your heart
Your eyes can tell stories and I love how you make me feel
I watch as you go about life, painting a masterpiece
One stroke at a time, coloring outside the lines, sometimes straying
    completely off course
But you never give up
And when I watch you, I feel like anything
```

Listing 12: Output crudo del modelo Mistral-7B LoRA a nivel de palabra.

## Conclusiones

El modelo Mistral-7B LoRA mostró un gran desempeño y marcada mejora en la coherencia frente a las arquitecturas tradicionales recurrentes y el resultado de TinyLLaMA. La pérdida promedio final fue de 1.59, lo que se traduce en una perplejidad de entrenamiento de 4.93, mientras que la PPL de validación se estimó entorno a 5.7. Estas cifras reflejan una generación estable, con poco sobreajuste, pese a realizarse solo en tres épocas, y el tamaño reducido del corpus.

A nivel cualitativo, las letras generadas demuestran una notable consistencia en cuanto a temática y carga emocional. El texto del prompt *“In the city lights I find myself”* se muestra como una canción con buena estructura lírica y estrofas, cerrando con un tono introspectivo *“Did your love bring out the best or the worst in me? Do you remember how I felt?”*.

La segunda canción, *“They say I’m broken but I’m breathing”* articula un discurso casi como de confesión, creo yo, imitando canciones como Cottonwood, Addict With A Pen

o Fake You Out de Twenty One Pilots. Finalmente, “*Sometimes my shadow sings louder than I do*” destaca por su tono poético y metáforas visuales, lo que sugiere que Mistral aprendió patrones estilísticos y temáticos complejos. Me parece que esto se ve impulsado por un buen prompt, que comparta el estilo y visión de la banda.

En comparación con TinyLLaMA-1.1B, Mistral produce versos más largos y mantiene una coherencia global más estable, aunque con una ligera pérdida de espontaneidad léxica. Creo que su ventana de atención amplia y el uso eficiente de LoRA permitieron modelar dependencias de largo alcance sin degradar la fluidez.

Algo que pienso es que, aunque entrenamos con aproximadamente el 1 % de los parámetros con LoRA, el resto del modelo, i.e. los 7 mil millones de parámetros de Mistral base, siguen determinando el comportamiento de la distribución lingüística general de los outputs. Es decir, LoRA no borra el conocimiento previo más bien lo modula a través de modificar un poco las matrices. Creo que por eso, sus salidas, aunque muy buenas e interesantes, no logran capturar por completo los clichés o estructuras más específicas de Twenty One Pilots. Es decir, al final sigue siendo un LLM generalista, que ajusta los sesgos de atención hacia patrones específicos.

En consecuencia, Mistral se adapta y se convierte en un híbrido de estilo global por su entrenamiento original y el estilo específico de mi corpus. Recordemos que por descomposición de LoRA:  $W' = W + BA$ . Donde  $B$  y  $A$  son matrices de bajo rango (entrenables con menos del 1 % de los params.). Entonces,  $W$  codifica todo el conocimiento general del modelo (con la gramática, semántica y razonamiento previo), y  $BA$  es un filtro que ajusta a los tokens y define los patrones del modelo sobre los que se dará prioridad.

Se intenta reorientar lo aprendido. LoRA “dobla” el espacio de atención, pero el punto de partida sigue siendo el embedding universal de Mistral. Esto, creo, explica por qué el modelo mantiene coherencia global y gramatical excelente, pero no siempre puede reproducir con fidelidad total el perfil artístico de Twenty One Pilots. Supongo que podríamos hacer más ajustes al finetuning para acercarnos cada vez más al estilo de Twenty One Pilots, pero eso se queda como trabajo futuro.

## Parte B | CNN

Usando la misma base de la tarea 1, entrena un modelo por cada arquitectura: CNN, RNN/LSTM/GRU, y un Transformer de Hugging Face con fine-tuning. Reporta accuracy y F1, incluye matriz de confusión y análisis de errores característicos.

Las CNN para texto modelan *n-gramas* como “filtros” que barren la secuencia de embeddings y extraen *features* locales robustas (p. ej., negación corta, intensificadores). El *max-over-time pooling* resume la activación más representativa por filtro, favoreciendo patrones discriminativos (*salient n-grams*).

### Arquitectura CNN

Sea  $X \in \mathbb{R}^{L \times d}$  la matriz de embeddings (longitud  $L$ , dimensión  $d$ ). Para un kernel 1-D de tamaño  $k$ :

$$c_i = \sigma(\langle W, X_{i:i+k-1} \rangle + b), \quad \mathbf{c} = [c_1, \dots, c_{L-k+1}], \quad \hat{\mathbf{c}} = \text{máx}(\mathbf{c})$$

donde  $W \in \mathbb{R}^{k \times d}$  y  $\sigma$  es ReLU. Con varios tamaños de kernel  $k \in \{2, 3, 4, 5\}$  y  $F$  filtros por tamaño, concatenamos  $\hat{\mathbf{c}} = [\hat{c}^{(k_1)}, \dots, \hat{c}^{(k_m)}]$  y proyectamos a  $K$  clases:

$$\hat{\mathbf{y}} = \text{Softmax}(W_o [\hat{\mathbf{c}}] + b_o), \quad \mathcal{L} = \text{CE}(\mathbf{y}, \hat{\mathbf{y}})$$

Los kernels capturan patrones locales ( $k=3-5$  suele alinear bien con bi/tri-gramas semánticos). El *global max-pool* selecciona la evidencia más fuerte por filtro, haciéndolo robusto a la posición. Con reseñas 1–5, las frases muy positivas/negativas emergen como activaciones altas; las clases intermedias (2–3–4) requieren matices y suelen confundirse si la capacidad o el regularizador no están calibrados.

La arquitectura que realizamos fue TextCNN: embeddings 256, kernels  $\{2, 3, 4, 5\}$  o  $\{3, 4, 5\}$ ,  $F \in \{64, 128\}$  filtros, *dropout* (0.5–0.6), AdamW (LR  $1e-3$  o  $2e-3$ ), *batch* 64, 20 épocas, *early stopping* por F1-macro en validación. El `train_cnn.py` guarda curvas de loss/F1, matriz de confusión y ROC OvR en el mejor punto, y usa `class_weights` desde `meta.json`: `contentReference[oaicite:1]index=1`

## Resultados

### Visualización de Modelos

Cuadro 5: TextCNN — Resumen de experimentos en MeIA (polaridad 1–5).

| Parámetro              | ..._f128_do60 | ..._f64_k345_do50 | ..._k345_f128_lr1e-3 |
|------------------------|---------------|-------------------|----------------------|
| <b>Kernels</b>         | 2,3,4,5       | 3,4,5             | <b>3,4,5</b>         |
| <b>Filtros</b>         | 128           | 64                | <b>128</b>           |
| <b>LR</b>              | 2e−3          | 2e−3              | 1e−3                 |
| <b>Dropout</b>         | 0.6           | 0.5               | <b>0.5</b>           |
| <b>Val F1-M</b>        | 0.414         | 0.425             | <b>0.456</b>         |
| <b>Test Acc / F1-M</b> | 0.374 / 0.357 | 0.410 / 0.379     | <b>0.448 / 0.441</b> |

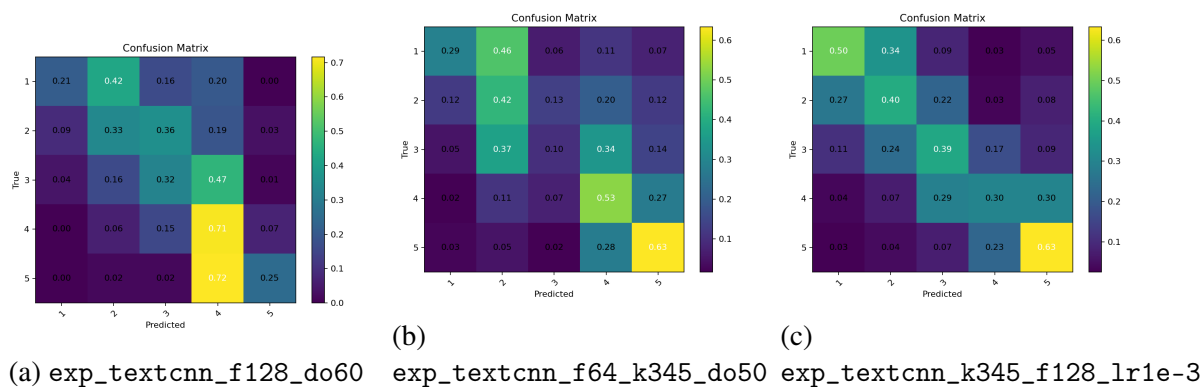


Figura 6: Matrices de confusión para los tres experimentos de TextCNN en el dataset MeIA. Muestran la clasificación de polaridad (1-5).

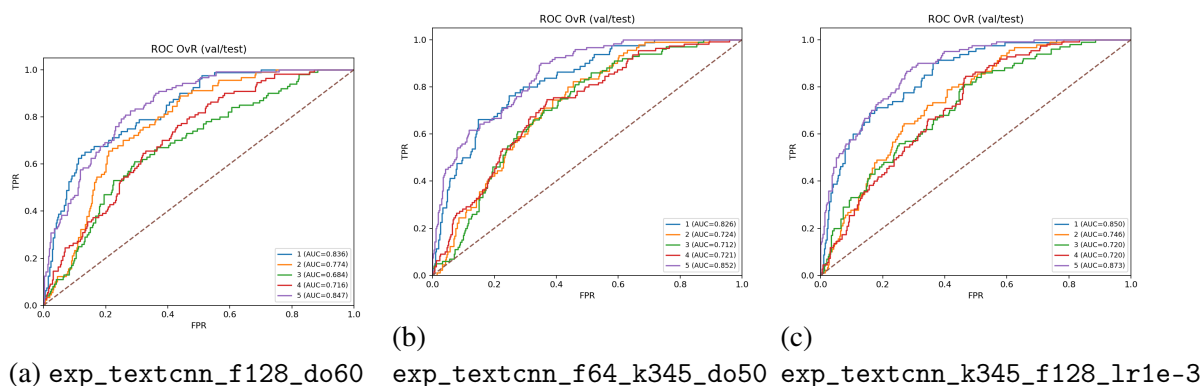


Figura 7: Curvas ROC One-vs-Rest (OVR) para los tres experimentos de TextCNN. Se evalúa la capacidad de discriminación entre cada clase de polaridad y el resto.

## Conclusiones

- Centrar los kernels en tamaños de 3 a 5 mejora el rendimiento en comparación con usar un rango más amplio (2 a 5), ya que captura patrones semánticos más “limpios”.
- Reducir el Learning Rate a  $1e-3$  estabilizó el entrenamiento en el conjunto de validación y produjo el mejor modelo generalizado (F1-macro en test = 0.441).
- Un valor de Dropout de 0.6 fue excesivo, ya que “acható” las distribuciones de probabilidad y degradó la métrica F1.
- Lamatriz de confusión muestra que los errores se concentran principalmente entre las clases adyacentes 2–3–4 (la frontera ordinal), mientras que las clases extremas (1 y 5) mantienen un buen *recall*.
- *Nota:* Las gráficas de Confusión y ROC OvR se generaron automáticamente al guardar el mejor checkpoint del modelo.

El modelo basado en Convolutional Neural Networks (TextCNN) demostró un desempeño sólido como línea base para la clasificación de polaridad en reseñas turísticas. La mejor configuración —con kernels centrados en 3, 4 y 5 palabras, 128 filtros por tamaño, tasa de aprendizaje de  $1 \times 10^{-3}$  y dropout de 0.5— alcanzó una precisión de 0.448 y un F1-macro de 0.441 en el conjunto de prueba, superando a las configuraciones previas con tasas de aprendizaje más altas o regularización excesiva. Se observó que el modelo captura con efectividad las señales extremas de polaridad, es decir, las reseñas claramente positivas

o negativas (clases 1 y 5), mientras que mantiene dificultades para distinguir los matices intermedios (clases 2, 3 y 4), un fenómeno atribuible a la naturaleza ordinal del problema y a la limitación inherente del pooling global en representar relaciones semánticas sutiles. Pese a ello, la TextCNN logra aprender representaciones discriminativas estables y generaliza adecuadamente al conjunto de validación, evidenciando que la optimización y la calibración del dropout son factores más determinantes que el aumento de filtros o la complejidad estructural. En síntesis, el modelo se consolida como un punto de partida robusto sobre el cual arquitecturas recurrentes o transformacionales podrán construir representaciones de contexto más finas en etapas posteriores del análisis.

## Parte B | RNN

Usando la misma base de la tarea 1, entrena un modelo por cada arquitectura: CNN, RNN/LSTM/GRU, y un Transformer de Hugging Face con fine-tuning. Reporta accuracy y F1, incluye matriz de confusión y análisis de errores característicos.

Recordemos que, a diferencia de las CNN, que capturan patrones locales posicionalmente invariantes, las RNN procesan el texto de manera secuencial, manteniendo un estado oculto que teóricamente puede codificar información de todo el contexto previo. Para tareas de clasificación, este enfoque permite modelar dependencias de largo alcance y el orden de las palabras.

Una mejora realizada para las RNN base son las RNN Bidireccionales, que procesa la secuencia en ambas direcciones, tanto de izquierda a derecha, y de derecha a izquierda. Esto permite que el estado oculto en cualquier punto contenga información tanto del contexto pasado como del futuro, resultando en representaciones más ricas.

## Arquitectura RNN

Para obtener un único vector que representa toda la secuencia, se aplica una operación de pooling sobre la secuencia de estados ocultos de la Bi-RNN. Si  $H = [h_1, h_2, \dots, h_L]$  es la secuencia de estados ocultos concatenados, donde cada  $h_t$  combina el estado hacia adelante y hacia atrás, la representación final  $h_{\text{pooled}}$  se obtiene como:

$$h_{\text{pooled}} = \text{Pool}(H)$$

El vector resultante  $h_{\text{pooled}}$  se pasa a una capa lineal final con una función de activación Softmax para la clasificación. La función de pérdida en este caso es la Entropía Cruzada, podnerada para manejar el desbalance de clases. Recordemos que en generación de texto, la métrica más usada de validación es la Perplejidad (PPL), que no es otra cosa más que la exponencial de la Entropía Cruzada.

$$\hat{y} = \text{Softmax}(W_o \cdot h_{\text{pooled}} + b_o) \quad L = \text{CrossEntropy}(y, \hat{y})$$

Los modelos con los que experimentamos partieron de una base con capa de Bi-RNN y MaxPooling. al observar un sobreajuste significativo, con diferencia porcentual de hasta 30 puntos de F1 entre training y validación, decidimos explorar diversas estrategias de regularización: aumento en el dropout, weight decay, label smoothing y diferentes estrategias

de pooling como mean y maxpooling+mean, para intentar encontrar una mejor generalización.

## Resultados

El primer modelo superó a la CNN, pero mostró un claro sobreajuste (Train F1  $\approx 0.78$  vs. Val F1  $\approx 0.47$ ). Los siguientes experimentos buscaron mejorar la regularización y explorar el impacto del mecanismo de *pooling*.

Cuadro 6: Bi-RNN — Resumen de experimentos en MeIA (polaridad 1–5).

| Parámetro       | ..._brnn_five (baseline) | ..._mean_do60 | ..._maxmean_ls002 |
|-----------------|--------------------------|---------------|-------------------|
| Pooling         | <b>max</b>               | mean          | max+mean          |
| Proj. Dropout   | <b>0.5</b>               | 0.6           | 0.5               |
| Weight Decay    | 1e−4                     | 2e−4          | 2e−4              |
| Label Smoothing | <b>0.0</b>               | 0.05          | 0.02              |
| Val F1-M        | <b>0.467</b>             | 0.430         | 0.451             |
| Test Acc / F1-M | <b>0.492 / 0.484</b>     | 0.462 / 0.444 | 0.464 / 0.440     |

## Visualización de Modelos

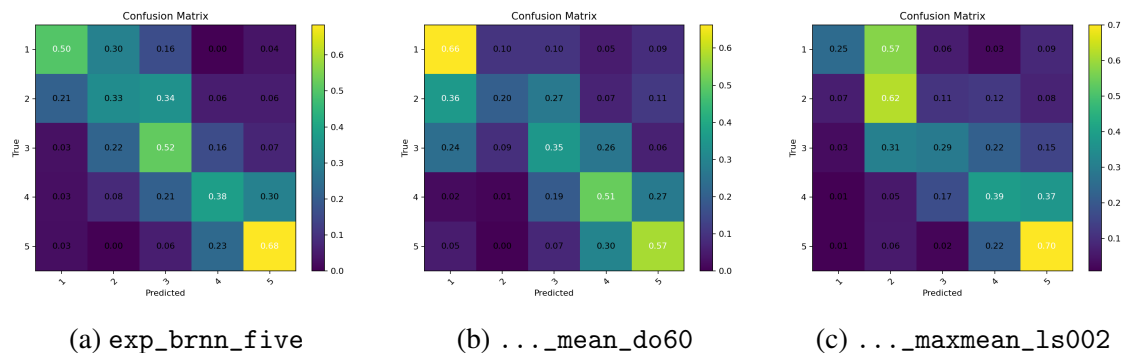


Figura 8: Matrices de confusión para los tres experimentos de Bi-RNN en el dataset MeIA. El modelo baseline (izquierda) muestra el mejor balance.



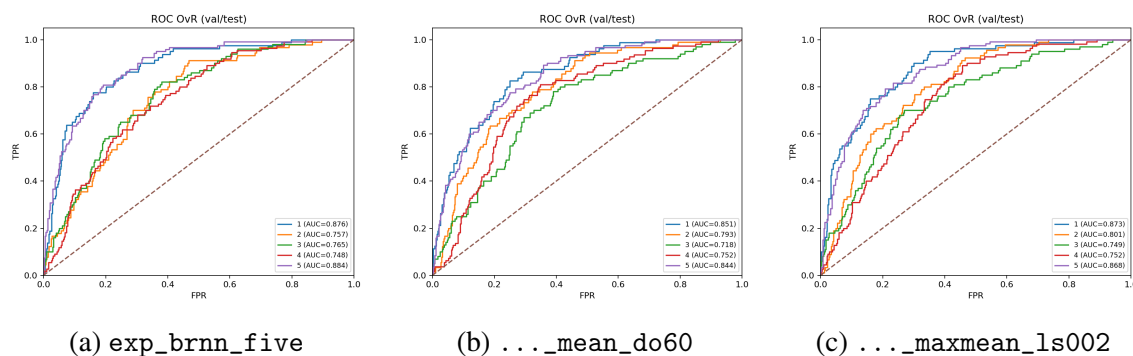


Figura 9: Curvas ROC One-vs-Rest (OVR) para los experimentos de Bi-RNN. Evalúan la capacidad de discriminación de cada clase.

## Conclusiones

- La arquitectura Bi-RNN con MaxPooling superó al mejor modelo de mi TextCNN, estableciendo una nueva línea base con un F1-macro de 0.484.
- El mecanismo de pooling resultó ser el factor más determinante, MaxPooling fue superior a MeanPooling y a la concatenación max+mean. Esto nos está indicando que para este problema las señales con alta carga semántica son más informativas que la evidencia contextual distribuida por el texto.
- Los intentos de regularización más agresiva, como label smoothing, o un mayor weight decay, fueron contraproducentes. Suavizar las predicciones parece diluir las distinciones sutiles entre las clases intermedias (2-3-4), empeorando el rendimiento general.
- Al igual que con la CNN, el desafío principal es la zona media de la polaridad, las clases intermedias ya mencionadas. Aunque el modelo captura bien los extremos (1 y 5), la naturaleza ordinal y la ambigüedad semántica de las puntuaciones intermedias son la principal causa de error.

Este modleo basado en una RNN bidireccional demostró ser más efectivo que la convolucional para capturar el contexto secuencial de las reseñas. La mejor configuración, una Bi-RNN con MaxPooling sobre los estados ocultos y una regularización moderada obtuvieron los mejores resultados de las pruebas, con un accuracy de 0.492, y un F1-Macro de 0.484 en el conjunto de prueba. El análisis de los experimentos revelaron que la estrategia de Pooling fue la que más eficientemente abordó el problema, el uso de MaxPooling logra

identificar características secuenciales más destacadas, lo cual fue benéfico para promediar la evidencia a lo largo de toda la secuencia, que tendía a centralizar las predicciones.

Pese a las mejoras, este modelo aún comparte el problema de la zona media, con las etiquetas de polaridad (2-3-4). La Bi-RNN se establece como una línea base sólida, confirmando que el modelado del orden secuencial puede aportar información de valor, y siendo un excelente punto de partida para evaluar el portencial de arquitecturas basadas en atención como los Transformers.

## Parte B | LSTM

Usando la misma base de la tarea 1, entrena un modelo por cada arquitectura: CNN, RNN/LSTM/GRU, y un Transformer de Hugging Face con fine-tuning. Reporta accuracy y F1, incluye matriz de confusión y análisis de errores característicos.

Si bien la arquitectura interna de la celda LSTM es la misma tanto para generación de texto como para clasificación de texto, el objetivo final, y la estructura del modelo, cambian.

### Arquitectura LSTM

En la tarea de generación, el modleo funciona como un Language Model. El objetivo es predecir el siguiente token de la secuencia  $y_t$ , dado el contexto de los tokens anteriores  $(x_1, x_2, \dots, x_n)$ .

Como se implementó en el código y arquitectura de la parte A, la salida LSTM en cada paso a través del tiempo  $h_t$  se pasa por una capa lineal de proyección, seguida de una Softmax sobre todo el vocabulario. La función de pérdida, como lo es la Entropía Cruzada, se calcula en cada token. De esa manera podemos hacer comparación de distribuciones de probabilidad predicah en el token real que sigue en la secuencia. El modleo se entrena para minimizar dicha pérdida a lo largo de la secuencia.

$$\hat{y}_t = \text{Softmax}(W_o h_t + b_o)$$

$$\mathcal{L}_{\text{generación}} = \sum_{t=1}^L \text{CrossEntropy}(\hat{y}_t, y_t)$$

El objetivo es aprender la distribución de probabilidad  $P(w_t | w_{1:t-1})$ .

### Un Modelo de Representación Global

Para clasificación de texto, no queremos predecir el siguiente token, queremos asignarle una etiqueta única a toda la secuencia o documento. Por lo tanto, no nos interesan las salidad de la LSTM a lo largo del tiempo, sino una Representación vectorial que resuma el contenido global de la secuencia completa.

El modelo procesa la secuencia de entrada, para de esa manera obtener los estados ocultos en cada paso,  $H = [h_1, h_2, \dots, h_L]$ . A partir de ahí, la diferencia clave es la introducción de una capa de Pooling.

- **Agregación de Información (Pooling):** en lugar de usar cada estado oculto  $h_t$ , se aplica una operación de pooling sobre todos los estados para obtener un único vector de tamaño fijo. Se pueden realizar diversos ajustes como la creación de un Mean-Pooling tal como vimos en RNN, pero el punto importante es que se toma un estado oculto final, que queda a modo de resumen de toda la secuencia que lo precede.
- **Proyección de Clases:** este vector resumen se pasa a la cabeza de la clasificación, que es una capa lineal final que proyecta el vector al número de clases deseadas, por ejemplo, 5 para la polaridad que tenemos de 1.0 a 5.0 estrellas.

$$\mathbf{v} = \text{Pool}(h_1, h_2, \dots, h_L)$$

$$\hat{\mathbf{y}}_{\text{clase}} = \text{Softmax}(W_{\text{clf}}\mathbf{v} + b_{\text{clf}})$$

La función de pérdida ('CrossEntropyLoss') se calcula una sola vez por secuencia, comparando la predicción final con la etiqueta verdadera de la secuencia.

$$\mathcal{L}_{\text{clasificación}} = \text{CrossEntropy}(\hat{\mathbf{y}}_{\text{clase}}, \mathbf{y}_{\text{clase}})$$

En resumen, mientras que el modelo generativo es un traductor secuencia-a-secuencia (de  $N$  tokens a  $N$  distribuciones de probabilidad), el modelo de clasificación es un codificador secuencia-a-vector, que condensa toda la información de la secuencia en una única representación para tomar una decisión final. Esta distinción es la que justifica el cambio en la arquitectura de salida y la forma en que se calcula la pérdida.

## Conclusiones

La primera implementación de una BiLSTM de una sola capa con *max-pooling* no solo superó a la RNN simple, sino que estableció un nuevo estado del arte para el problema, alcanzando un F1-macro en test superior a 0.51. Sin embargo, este éxito vino acompañado de un **\*\*sobreajuste extremo\*\***, con métricas en el conjunto de entrenamiento que superaban el 90

Cuadro 7: BiLSTM — Resumen de experimentos en MeIA (polaridad 1–5), transpuesto.

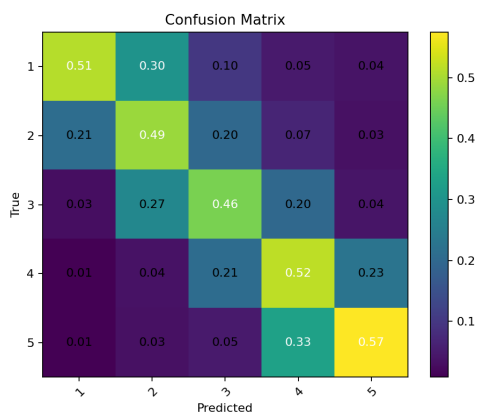
| Parámetro       | ..._bilstm_max_lr1e-3 | ..._bilstm4_max_do20 | ..._bilstm_maxmean... | ..._bilstm_max_regstrong |
|-----------------|-----------------------|----------------------|-----------------------|--------------------------|
| Capas LSTM      | 1                     | 2                    | 2                     | 1                        |
| Pooling         | max                   | max                  | max+mean              | max                      |
| Regularización  | Base                  | Base                 | Base                  | Fuerte (D.O., WD, W.D.)  |
| Val F1-M        | 0.495                 | 0.493                | 0.489                 | <b>0.505</b>             |
| Test Acc / F1-M | <b>0.514 / 0.515</b>  | 0.480 / 0.461        | 0.474 / 0.470         | 0.484 / 0.477            |

## Conclusiones

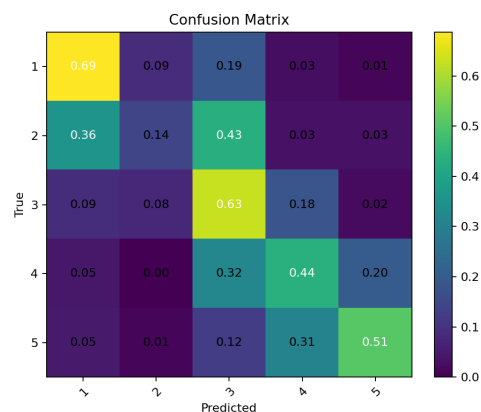
- El modelo baseline de BiLSTM de una capa se consolidó como el de mejor rendimiento de todas las arquitecturas recurrentes y convolucionales, alcanzando un F1-macro de 0.515 en el conjunto de prueba.
- El sobreajuste fue el principal desafío. A pesar de los intentos por mitigarlo, ningún modelo regularizado superó al baseline en el conjunto de test, aunque sí lograron reducir la brecha entre las métricas de entrenamiento y validación.
- Aumentar la profundidad del modelo (usando 2 capas de BiLSTM) resultó contraproducente. Incrementó el tiempo de cómputo y la complejidad sin una mejora en la generalización, de hecho, el rendimiento en test disminuyó.
- La estrategia de regularización fuerte (mayor dropout, weight decay y word dropout) logró el mejor F1-macro en el conjunto de validación (0.505), indicando que el modelo estaba generalizando mejor a ese subconjunto de datos. Sin embargo, esta mejora no se tradujo al conjunto de test, donde el rendimiento fue inferior al del baseline.
- Se reitera la conclusión de secciones anteriores: el MaxPooling sigue siendo la estrategia de agregación más efectiva para este corpus, superando a la combinación max+mean.

La arquitectura BiLSTM demostró ser la más poderosa hasta el momento, con un modelo simple de una sola capa estableciendo un nuevo benchmark de rendimiento. El análisis de los experimentos demuestra que el modelo más simple, a pesar de un sobreajuste masivo en el entrenamiento, encontró un mínimo en el espacio de parámetros que generalizó de forma más efectiva al conjunto de prueba final. Las intervenciones lógicas para mejorar el modelo, aumentar su capacidad con más capas o forzar la generalización con una regularización más estricta, paradójicamente degradaron su rendimiento en la evaluación final. Esto indica que las técnicas de regularización, si bien controlaron el sobreajuste, pudieron haber impedido al modelo aprender ciertos patrones cruciales presentes en el corpus, o que el conjunto de validación no fue un proxy perfecto para el de test. A pesar de todo, la BiLSTM se establece como el mejor modelo de redes neuronales hasta el momento.

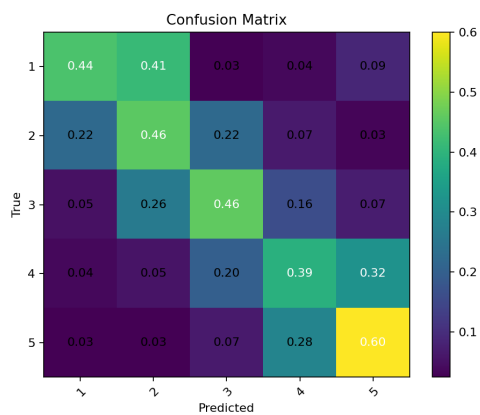
## Visualización de Modelos



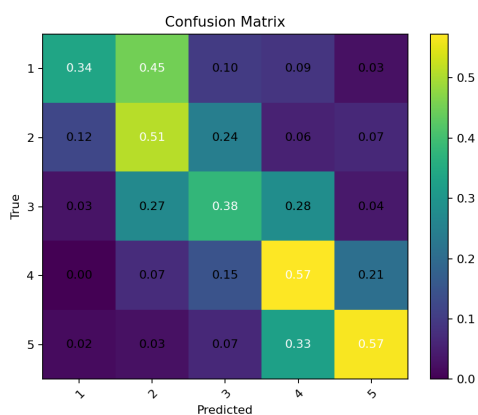
(a) ...max\_lr1e-3



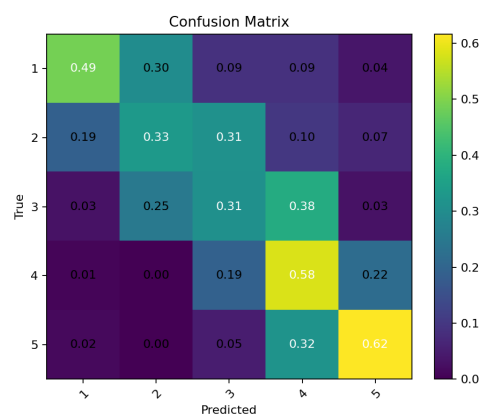
(b) ...bilstm4\_max...



(c) ...maxmean...



(d) ...max\_regstrong



(e) ...h192\_do60...

Figura 10: Matrices de confusión para los cinco experimentos de BiLSTM. El modelo baseline (primera imagen, arriba a la izquierda) muestra el mejor rendimiento.

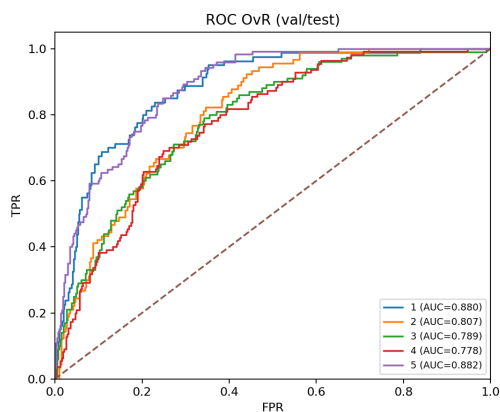
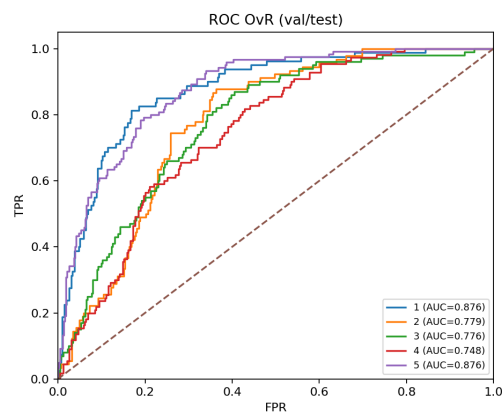
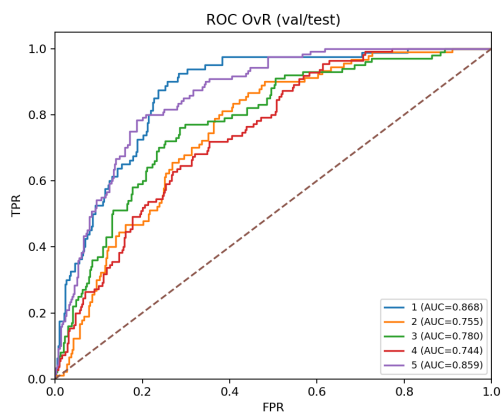
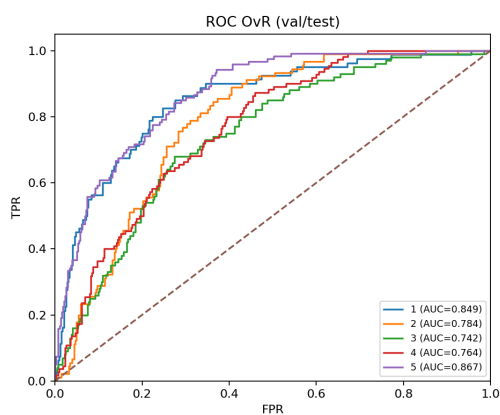
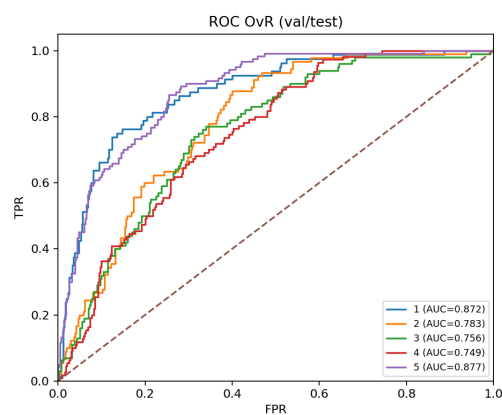
(a) `...max_lr1e-3`(b) `...bilstm4_max...`(c) `...maxmean...`(d) `...max_regstrong`(e) `...h192_do60...`

Figura 11: Curvas ROC One-vs-Rest (OVR) para los cinco experimentos de BILSTM. Se evalúa la capacidad de discriminación de cada clase.

## Parte B | GRU

Usando la misma base de la tarea 1, entrena un modelo por cada arquitectura: CNN, RNN/LSTM/GRU, y un Transformer de Hugging Face con fine-tuning. Reporta accuracy y F1, incluye matriz de confusión y análisis de errores característicos.

Ya hemos introducido a la Gated Recurrent Unit (GRU) como una alternativa eficiente a la LSTM. Su diseño busca resolver los mismos problemas de desvanecimiento de gradiente y gestión de memoria a largo plazo, con una arquitectura más simple y menos parámetros, lo que puede resultar en tiempos de entrenamiento más económicos y un rendimiento comparable a la LSTM.

Para la tarea de clasificación de texto, la celda GRU se integra en la misma arquitectura de alto nivel que la LSTM. El modelo procesa las secuencias de entrada con una o más capas BiGRU para generar secuencias de estados ocultos. Posteriormente, una capa de pooling, ya sea con MaxPooling o MeanPooling, agrega los estados a un único vector de representaciones, que finalmente es proyectado por una capa lineal para obtener las probabilidades de cada clase.

## Resultados

Se evaluó la arquitectura BiGRU como una alternativa computacionalmente más eficiente a la BiLSTM. El experimento inicial, utilizando una configuración análoga al baseline de la LSTM, arrojó un resultado inesperado: con un F1-macro de 0.468, su rendimiento fue inferior no solo al de la LSTM, sino también al de la RNN simple. Este punto de partida motivó una serie de experimentos para determinar si una regularización más ajustada o cambios en la capacidad del modelo podrían mejorar su desempeño.

Cuadro 8: BiGRU — Resumen de experimentos en MeIA (polaridad 1–5).

| Parámetro       | ..._bigru_max_lr1e-3 | ..._bigru_max_regstrong | ..._bigru_h192_regstrong | ..._bigru2_max_do20 |
|-----------------|----------------------|-------------------------|--------------------------|---------------------|
| Hidden Size     | 256                  | <b>256</b>              | 192                      | 256                 |
| Capas GRU       | 1                    | <b>1</b>                | 1                        | 2                   |
| Regularización  | Base                 | <b>Fuerte</b>           | Fuerte                   | Base                |
| Val F1-M        | 0.485                | 0.492                   | <b>0.502</b>             | 0.479               |
| Test Acc / F1-M | 0.476 / 0.468        | <b>0.498 / 0.491</b>    | 0.480 / 0.467            | 0.478 / 0.464       |

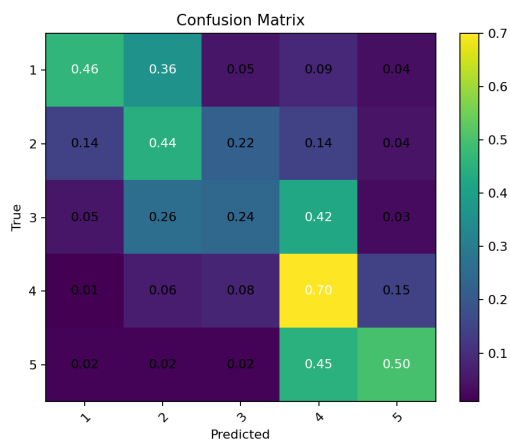


## Conclusiones

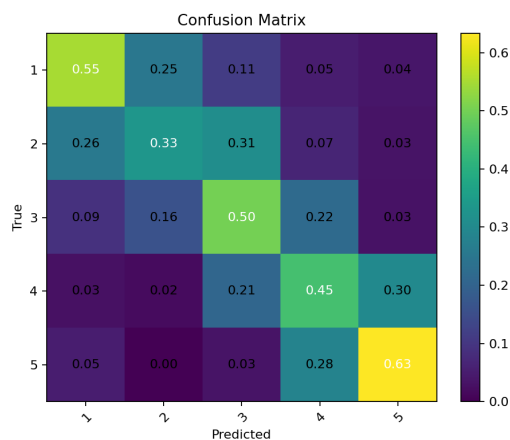
- La arquitectura BiGRU demostró ser competitiva, pero no logró superar el rendimiento de la BiLSTM. El mejor modelo alcanzó un F1-macro de 0.491.
- A diferencia de la BiLSTM, donde la regularización fuerte fue perjudicial, en la BiGRU fue esencial para el buen rendimiento. El modelo con regularización agresiva (mayor dropout, word dropout, etc.) superó significativamente al baseline, sugiriendo que la arquitectura GRU se beneficia más de estas técnicas en este problema.
- El modelo con la mejor métrica en el conjunto de validación ( $F1 = 0.502$ , . . . `h192_regstrong`) no fue el mejor en el conjunto de test. Este patrón, ya observado con la LSTM, subraya la importancia de la evaluación final en el conjunto de prueba.
- Incrementar la profundidad del modelo apilando dos capas de BiGRU fue, una vez más, una estrategia ineficaz que degradó el rendimiento final.

La BiGRU se posiciona como una arquitectura recurrente sólida, cuyo rendimiento final se sitúa entre la RNN simple y la más compleja BiLSTM. Estos resultados demuestran la superioridad de la LSTM: mientras que el modelo baseline fue decepcionante, una estrategia de regularización fuerte fue la clave para desbloquear su potencial, logrando un F1-macro de 0.491. Esto sugiere que la estructura interna más simple de la GRU podría ser más propensa al sobreajuste en este dataset, un problema que fue mitigado eficazmente con las técnicas de regularización aplicadas. Aunque su eficiencia computacional la convierte en una opción atractiva, en términos de rendimiento puro en esta tarea, la BiLSTM mantiene la superioridad. Con esto se concluye la exploración de arquitecturas recurrentes, habiendo establecido una jerarquía clara de rendimiento:  $\text{BiLSTM} > \text{BiGRU} > \text{Bi-RNN} > \text{TextCNN}$ .

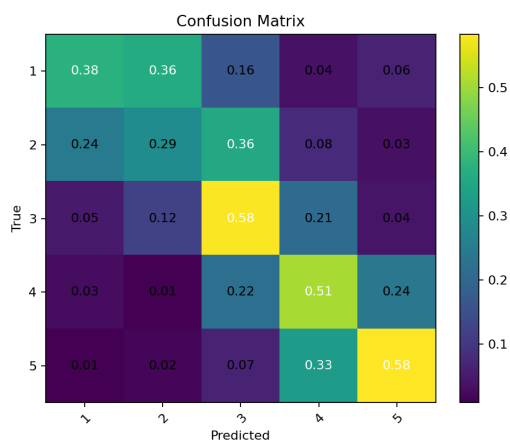
## Visualización de Modelos



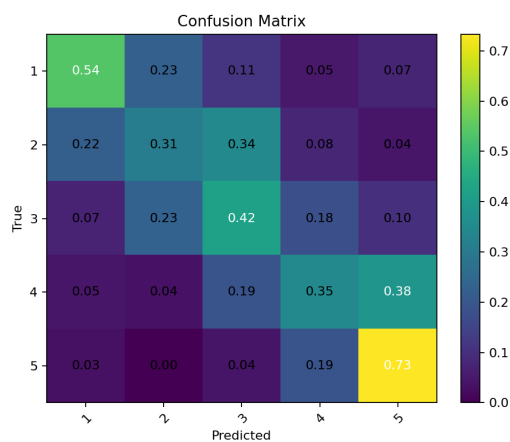
(a) ...max\_lr1e-3



(b) ...max\_regstrong

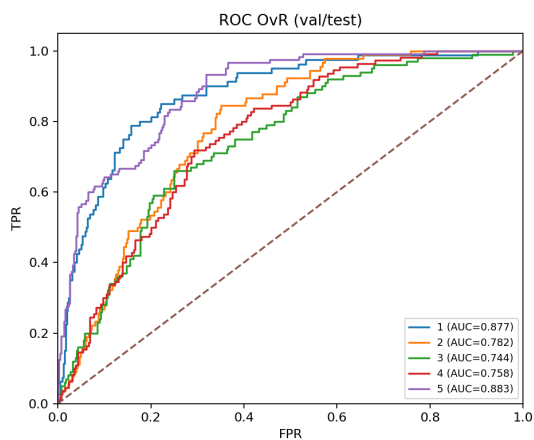


(c) ...h192\_regstrong

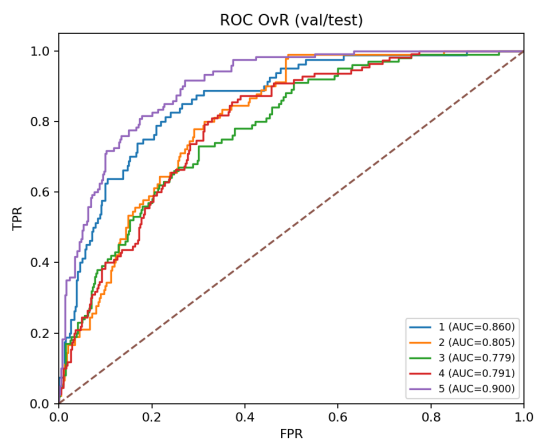


(d) ...bigru2\_max...

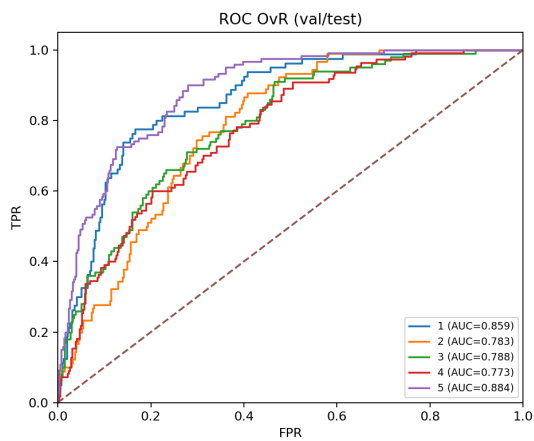
Figura 12: Matrices de confusión para los experimentos de BiGRU. El modelo con regularización fuerte (arriba, derecha) logró el mejor rendimiento en test.



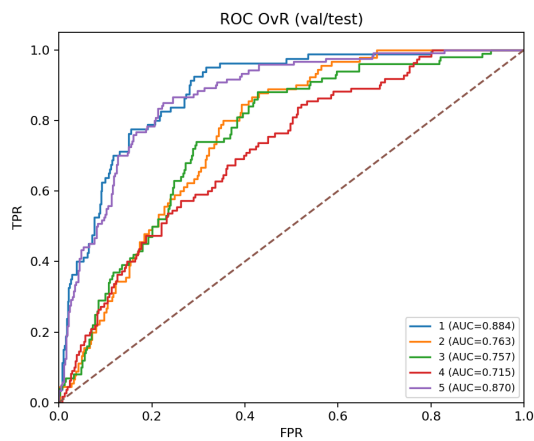
(a) ...max\_lr1e-3



(b) ...max\_regstrong



(c) ...h192\_regstrong



(d) ...bigru2\_max...

Figura 13: Curvas ROC One-vs-Rest (OVR) para los experimentos de BiGRU.

## Tranformers para clasificación de textos

Ya hemos hablado acerca de que la arquitectura Transformer representó una ruptura fundamental con los modelos recurrentes y convolucionales tradicionales dentro del Deep Learning. Eliminar la dependencia explícita de la secuencialidad y cambiarla por un mecanismo llamado “auto-atención” que permite procesar todos los tokens en paralelo, volvió a esta arquitectura en toda una revolución dentro del NLP, y el DL en general.

En lugar de propagar los estados ocultos de manera recurrente como en las RNN o los LSTM/GRU, el Transformer calcula relaciones directas entre todas las posiciones de una secuencia mediante pesos de atención que cuantifican la relevancia contextual de cada token respecto a los demás. Matemáticamente, estos trabajan con tres proyecciones que ya mencionamos en la sección pasada de Transformers, pero para el caso generativo, las matrices de consulta  $Q$ , clave  $K$  y valor  $V$ :

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

donde  $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ , son matrices de parámetros. La atención para cada token  $i$  se calcula como una combinación ponderada de los valores  $V$ , donde los pesos derivan de una similitud escalar entre las consultas y las claves:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Este cálculo define la contribución de cada token al contexto del token actual en la secuencia. Al combinar varias cabezas de atención, el modelo puede capturar simultáneamente distintos tipos de relaciones tanto semánticas como sintácticas.

## Clasificación de texto con Transformers

En tareas de clasificación, como lo es análisis de sentimiento, categorización temática o detección de polaridad, el Transformer se usa como codificador contextual. Cada token produce un embedding contextualizado  $h_i$ , y un vector de resumen de la secuencia se pasa a una capa densa para estimar la probabilidad de la clase:

$$\hat{y} = \text{Softmax}(W_c h_{[\text{CLS}] + b_c})$$

La función de pérdida por excelencia es la entropía cruzada entre la distribución predicha  $\hat{y}$  y la verdadera  $y$ :

$$\mathbb{L}_{CE} = - \sum_{k=1}^K y_k \log \hat{y}_k$$

Durante el fine-tuning, las capas del transformer se pueden actualizar de manera parcial o total, acorde al tamaño del corpus y los recursos disponibles. En la práctica, modelos como BETO, BERT-base o DistilBERT logran estados del arte en clasificación textual, esto gracias a sus capacidad de capturar relaciones semánticas a largo plazo sin recurrencia explícita.

## Contraste con el enfoque generativo

La distinción entre el uso generativo y discriminativo de los Transformers radica, principalmente, en el objetivo del entrenamiento y el flujo de la información. Podemos ver esto en la siguiente tabla:

Cuadro 9: Comparación entre modelos de lenguaje generativos y discriminativos.

| Aspecto          | Generativo (GPT, LLaMA)        | Discriminativo (BERT, BETO)             |
|------------------|--------------------------------|-----------------------------------------|
| Tipo de atención | Causal (unidireccional)        | Bidireccional                           |
| Objetivo         | Predicción del siguiente token | Predicción de tokens                    |
| Salida           | Texto continuo                 | Etiqueta de clase                       |
| Métrica típica   | Perplejidad (PPL)              | Accuracy, F1-score                      |
| Uso típico       | Generación de texto, diálogo   | Clasificación, análisis de sentimientos |

En la práctica, ambos modelos comparten la misma base estructural, bloques de atención, normalización y proyecciones lineales, pero difieren en cómo restringen el acceso al contexto y en el tipo de tarea supervisada que se les plantea. Mientras que los modelos generativos como GPT o LLaMA se entrenan para predecir la próxima palabra en la secuencia, los modelos tipo BERT/BETO internalizan el contexto completo para producir embeddings discriminativos que pueden ser reutilizados para tareas de fine-tuning eficiente. Este cambio de paradigma logró unificar, en una misma arquitectura, el aprendizaje de representaciones lingüísticas, siendo así el estado del arte actual y con lo que habrían soñado los fundadores de esta ciencia.

## Parte B | BETO

Usando la misma base de la tarea 1, entrena un modelo por cada arquitectura: CNN, RNN/LSTM/GRU, y un Transformer de Hugging Face con fine-tuning. Reporta accuracy y F1, incluye matriz de confusión y análisis de errores característicos.

Para esta prueba reutilizamos lo usado en RESTMex2025.

Es claro como un transformer mejora sustancialmente los resultados, superando por bastante a las arquitecturas de redes neuronales recurrentes. La jerarquía ahora queda establecida de manera muy clara: Transformers » BiLSTM > BiGRU > Bi-RNN > TextCNN.

## Visualización de Modelos

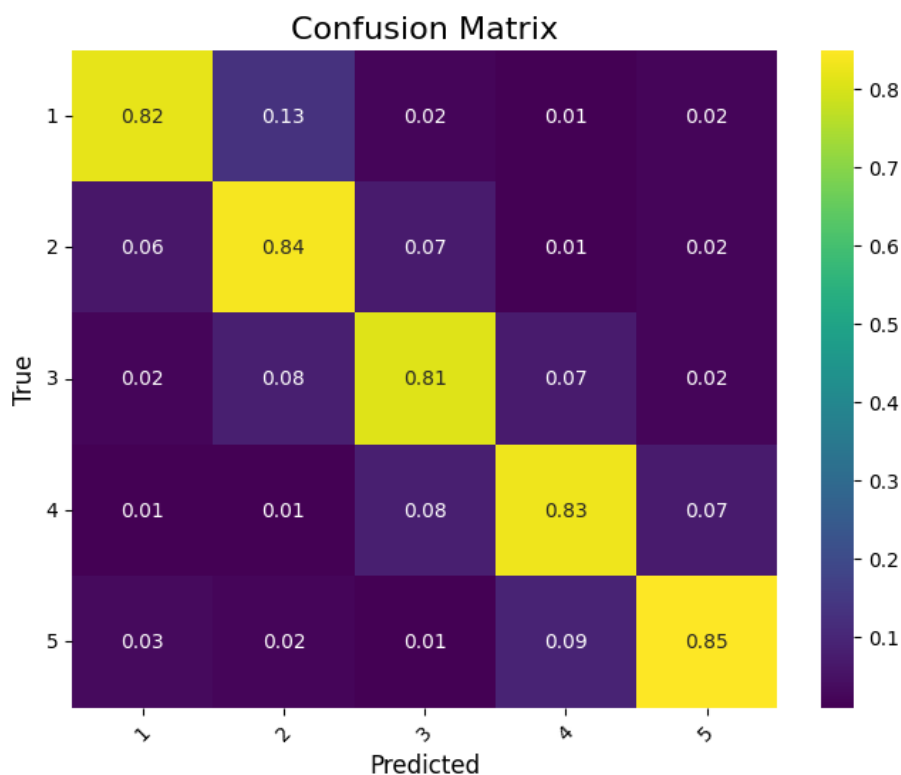


Figura 14: Matriz de Confusión | Polaridad Transformer.